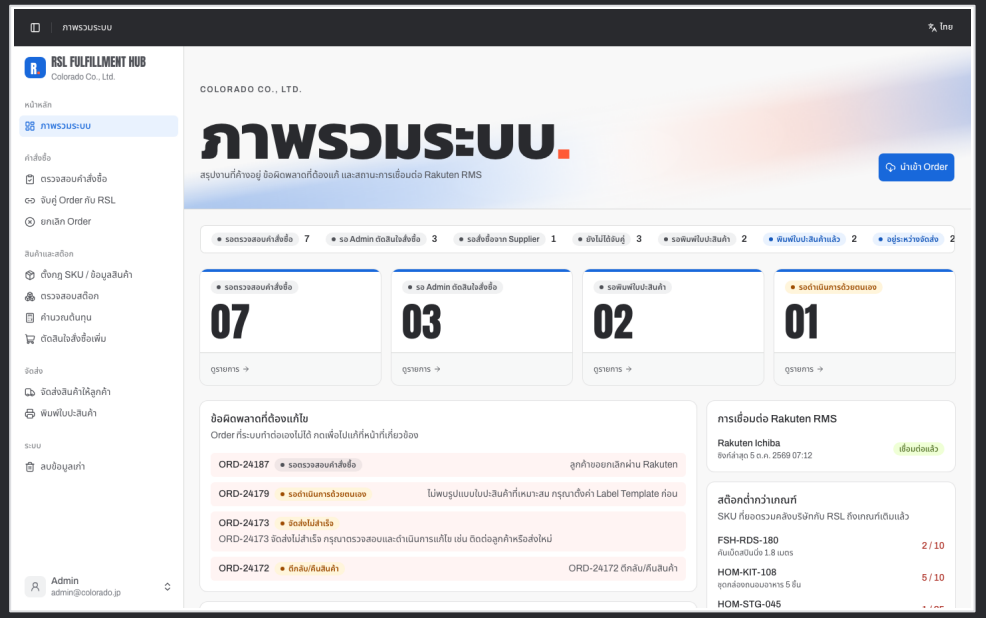
Colorado Back-Office System.

ระบบจัดการหลังบ้านอัตโนมัติ เฟส 1
Rakuten Ichiba, Rakuten RMS และ RSL



ระบบอัตโนมัติ 3 เรื่อง

- Auto label
- Auto reorder
- Unit cost calculator

ผู้จัดทำ

นายธนภัทร พูลโภคา
6710450911

นายอชิระ ภู่รุ่งเรือง
6710451488

นางสาวปิยากร ผลพานิช
6710451054

Agenda 3 ช่วงของการนำเสนอ

01

ธุรกิจและ Business Process

สไลด์ 1–14

- แนะนำธุรกิจ (English)
- ปัญหาและวัตถุประสงค์
- Business Process เดิมและใหม่
- Use Case Diagram และ CRUD

02

Use Case และ Sequence

สไลด์ 15–33

- Use Case ทั้ง 17 ตัว
- Sequence แบบ MVC + Collaboration
- 3S, 5A, 6A, 7S แบบละเอียด
- กลุ่ม Use Case ที่เหลือ

03

ออกแบบฐานข้อมูลและเว็บจริง

สไลด์ 34–57

- ER, ตาราง, Class
- State ของทุกข้อมูลที่ถูกแก้ไข
- สถาปัตยกรรม, SQL, การทดสอบ
- สาธิตเว็บ + SELECT สรุป

ตัวเลขทั้งหมดในสไลด์ตรงกับรายงานฉบับสุดท้าย 17 Use Case · 17 ตาราง · SQL 113/113 รันผ่าน · ทดสอบ 36/36 กรณีผ่าน

Business introduction แนะนำธุรกิจ

One owner handles every order by hand.

Market	Rakuten Ichiba, Japan
Products	Fishing lures, outdoor goods, Thai herbal inhalers
Since	2009, started as a side business
Logistics	Rakuten Super Logistics (RSL) stores and ships
Next	Yahoo! Auctions and Amazon

One order today

Customer orders on Rakuten Ichiba

↓

Owner checks stock in two warehouses — by hand

↓

Owner matches the order with RSL — by hand

↓

Owner formats and prints the label — by hand

↓

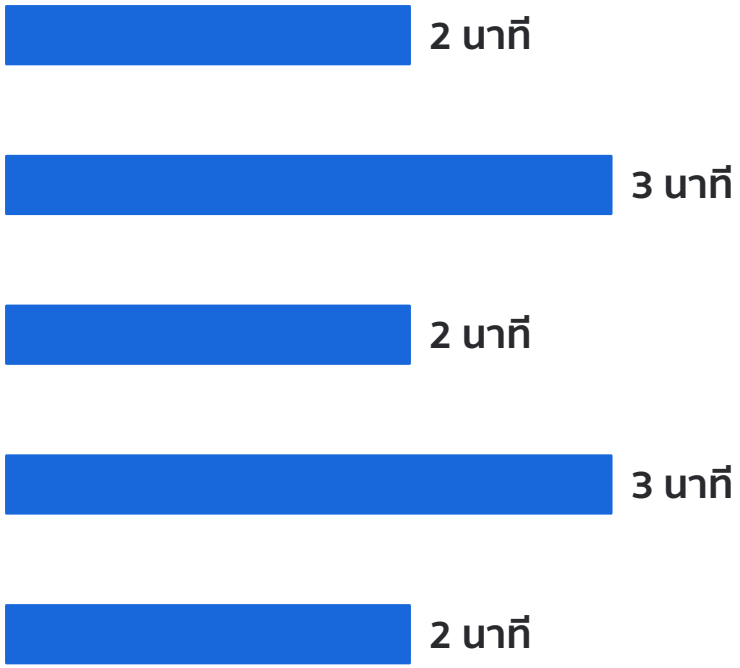
RSL packs, delivery company ships

↓

Owner sends the tracking number — by hand

Problems งานมือที่เกิดซ้ำทุก Order

นาทีต่อ 1 Order (ตารางที่ 1-1)



รวม 12 นาที/Order × 5–10 Order ต่อวัน = 60–120 นาที

ค่าประมาณจากขั้นตอนที่เจ้าของเล่า ยังไม่ได้จับเวลาจริง

1–2 ชม./วัน

งานมือหลังบ้าน (ประมาณ) ≈ 30–60 ชม./เดือน

400 ¥

ส่งเองแพงกว่าคู่แข่งต่อชิ้น (600 ¥ เทียบ 200 ¥)

20–30 นาที

รวมต้นทุน 8 รายการเองเพื่อหาต้นทุนจริง 1 SKU

Objectives ระบบอัตโนมัติ 3 เรื่อง และตัวชี้วัด

01

Auto label

จัดรูปแบบใบปะหน้าทีเมื่อรู้แหล่งส่ง
เจ้าของเหลือแค่กดพิมพ์

Use Case 3S · 1S · 6S · 9A

02

Auto reorder

เช็กสต๊อก 2 คลัง แจ้ง Order ที่ของไม่พอ
เตรียมใบสั่งซื้อให้เจ้าของอนุมัติ

Use Case 3S · 5A · 6A

03

Unit cost

คำนวณต้นทุนต่อหน่วยจาก 8 รายการ
สูตรเดียวกันทุกครั้ง พร้อมเก็บประวัติ

Use Case 4S

ตัวชี้วัดภายใน 3 เดือนหลังเปิดใช้ (1.3)

ทุก Order ใหม่

ได้ใบปะถูกต้องโดยไม่จัดรูปแบบเอง

ทุก SKU

ที่สต๊อกต่ำกว่าเกณฑ์ถูกแจ้งเตือน

≤ 1 นาที

คำนวณต้นทุนต่อหน่วย
(เดิม 20–30 นาที)

≥ 50%

เวลางานหลังบ้านต่อวันลดลง

Scope เฟส 1 และสิ่งที่ยังให้คนตัดสินใจ

อยู่ในเฟส 1

- Rakuten Ichiba, Rakuten RMS และ RSL เท่านั้น
- คลัง 2 แห่ง: RSL และคลังของบริษัท
- ผู้ใช้หลักคือเจ้าของ (Super Admin)
- ต่อระบบภายนอกผ่านตัวเชื่อม (adapter) ตอนนี้เป็นตัวจำลอง

นอกขอบเขต

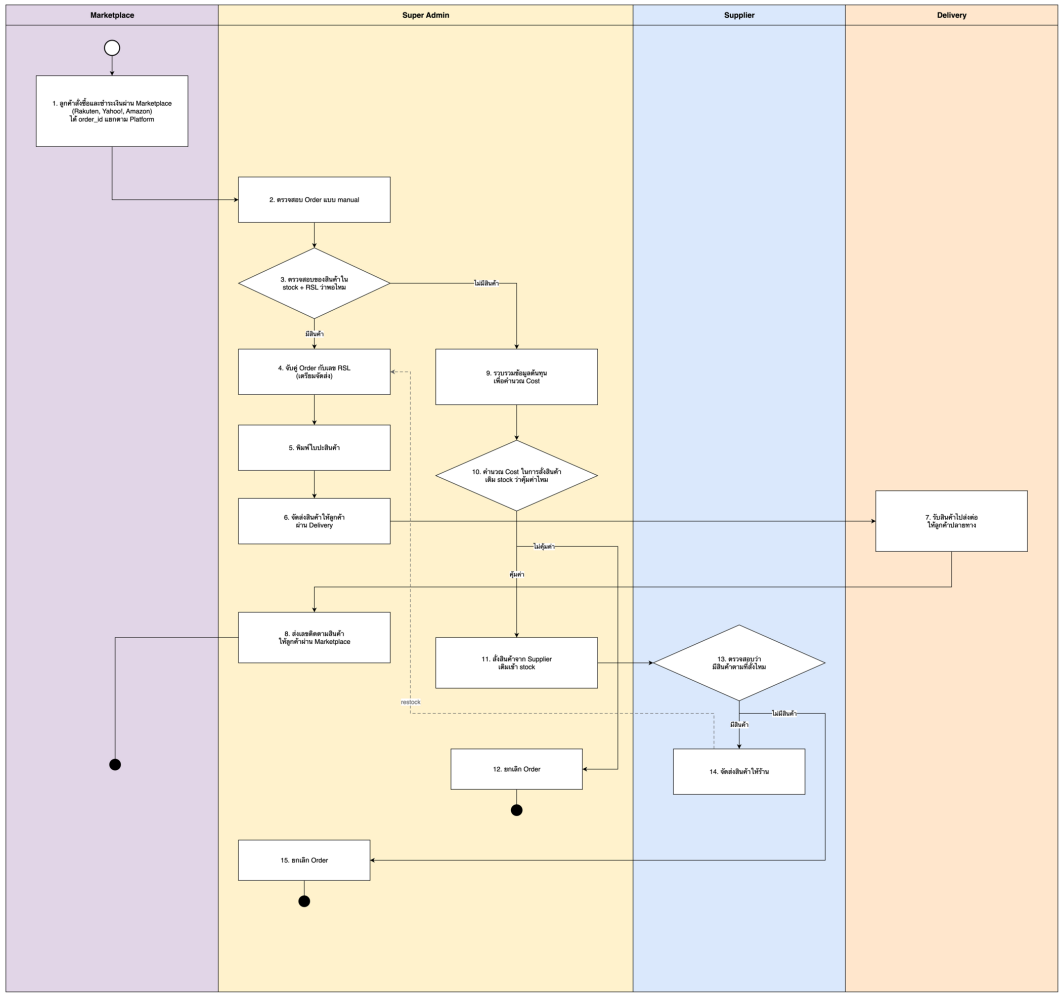
- Yahoo! Auctions, Amazon (เฟสถัดไป)
- รับชำระเงิน และสื่อสารกับลูกค้าโดยตรง
- ยืนยันตัวตน 2 ชั้น (2FA) ของบัญชี Admin
- Mercari Shops (เลิกใช้แล้ว)

Human-in-the-loop: คนตัดสินใจขั้นสำคัญ

ระบบทำงานซ้ำให้ แต่ขั้นที่ใช้เงินจริง เจ้าของต้องกดเอง

1. **5A ตัดสินใจคุ้มค่าหรือไม่**
   ระบบแสดงต้นทุนล่าสุดและราคาขาย แต่ไม่ตัดสินใจแทน
2. **6A ส่งใบสั่งซื้อและรับสินค้า**
   เจ้าของยืนยันการสั่งซื้อ และกดรับของเมื่อ Supplier ส่งมาถึง
3. **7A, 10A ยกเลิกและลบข้อมูล**
   ต้องใส่เหตุผล และเก็บสำเนาข้อมูล (snapshot) ก่อนลบเสมอ

Business process: As-Is Super Admin ทำทุกขั้นเอง



15 ขั้นตอน (2.2.1–2.2.15)

1 รับ Order และตรวจเอง

ลูกค้าสั่งผ่าน Marketplace
Super Admin ตรวจเลข Order ทีละรายการ

2 เช็กสต๊อกและจับคู่ RSL เอง

ถ้าพอ จับคู่เลข RSL พิมพ์ Label
ส่งมอบ Delivery แล้วแจ้งเลขติดตาม

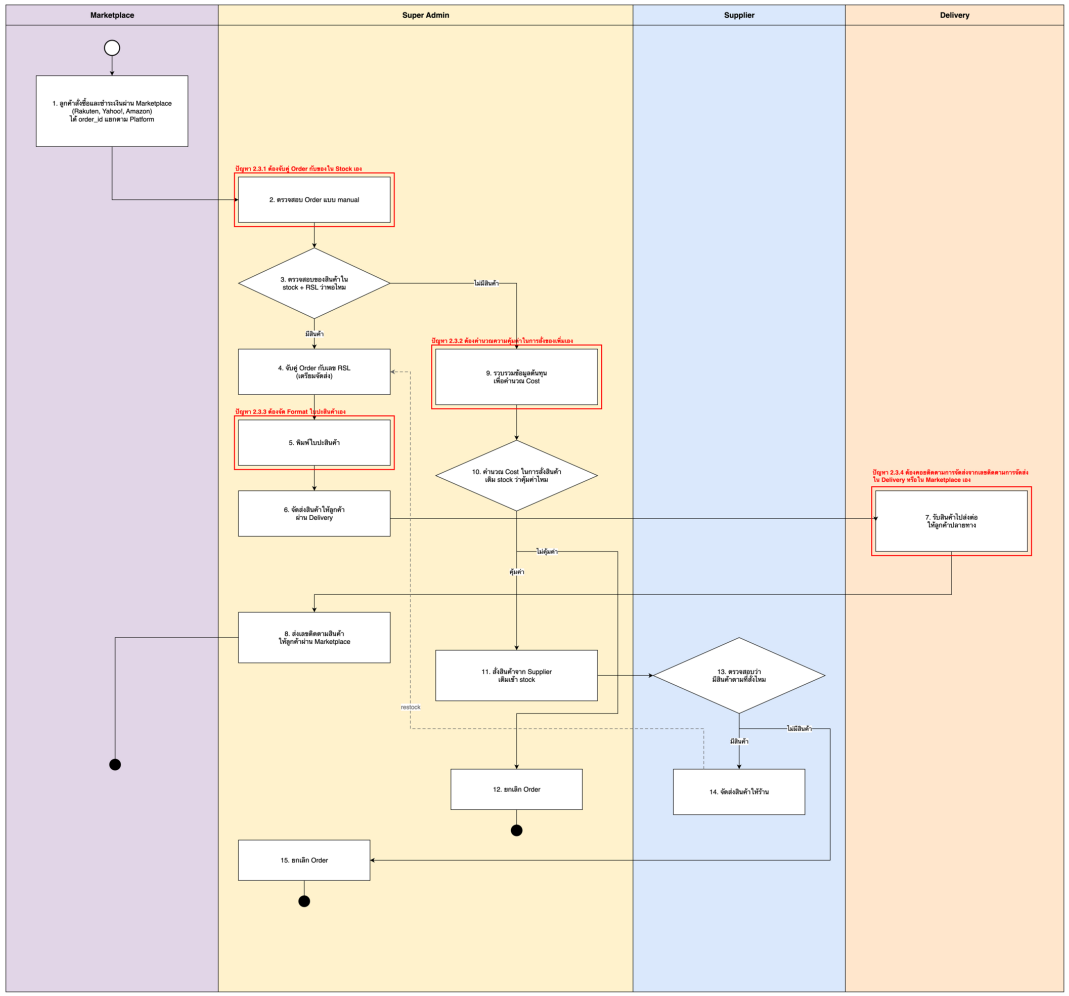
3 สต๊อกไม่พอ คำนวณต้นทุนเอง

คุ้มค่า → สั่ง Supplier
ไม่คุ้มค่า หรือ Supplier ไม่มีของ → ยกเลิก Order

4 Supplier ส่งของ วนกลับไปจับคู่ RSL

เติมสต๊อก แล้วเริ่มจับคู่ใหม่

Pain points 4 จุดที่ระบบใหม่ต้องแก้



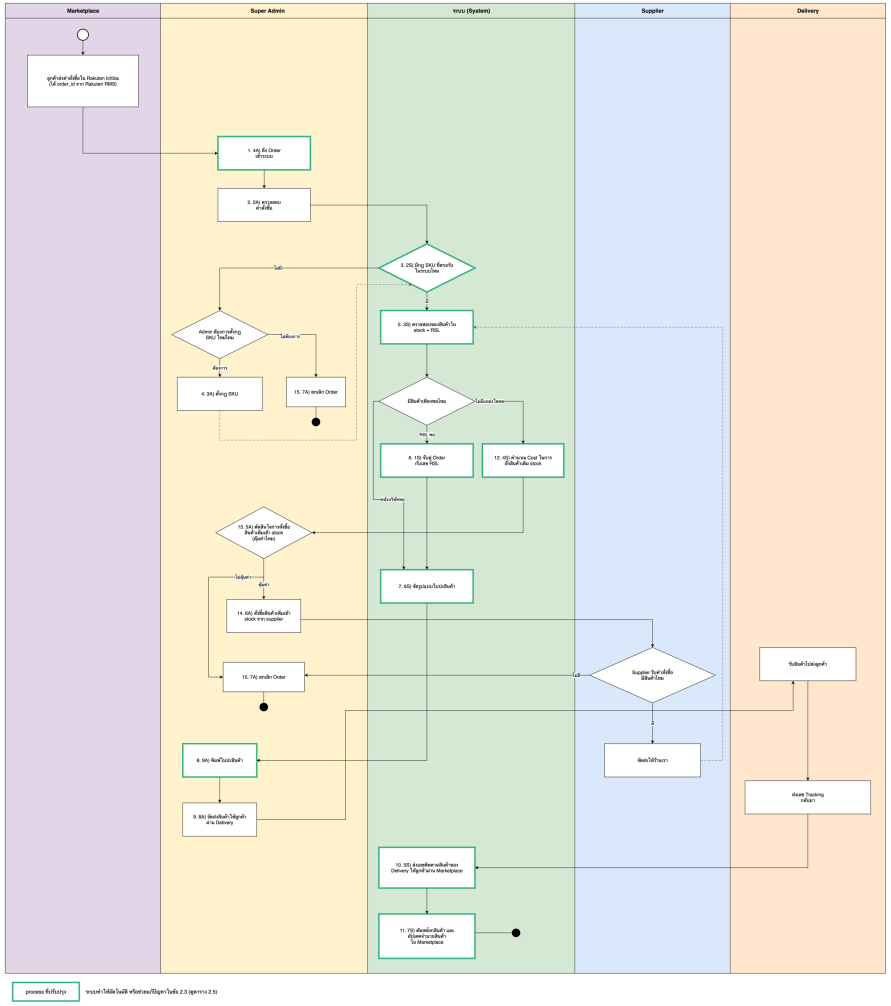
1. **จับคู่ Order กับสต๊อกเอง**
   ประมาณ 5 นาที/Order หรือ 25–50 นาที/วัน (2.3.1)

2. **คำนวณความคุ้มค่าเอง**
   รวม 8 รายการ ประมาณ 20–30 นาที/ครั้ง (2.3.2)

3. **จัดรูปแบบใบปะเอง**
   ประมาณ 3 นาที/Order หรือ 15–30 นาที/วัน (2.3.3)

4. **ติดตามพัสดุและแจ้งลูกค้าเอง**
   ประมาณ 2 นาที/Order หรือ 10–20 นาที/วัน (2.3.4)

Business process: To-Be เพิ่มช่อง (lane) ของระบบ: ระบบรับงานซ้ำ คนตัดสินใจ



ระบบทำเอง

จับคู่กฎ SKU · เลือกแหล่งสต๊อก · จับคู่ RSL
คำนวณต้นทุน · จัดรูปแบบใบปะ
แจ้งเลขติดตาม · ตัดสต๊อก

1S – 7S

คนยังทำ

นำเข้า Order · ตรวจคำสั่งซื้อ · ตั้งกฎ SKU
อนุมัติและสั่งซื้อ · รับของ · ยกเลิก
พิมพ์ใบปะ · ส่งมอบพัสดุ

4A 2A 3A 5A 6A 7A 8A 9A

เส้นทางใหม่หลัง 3S

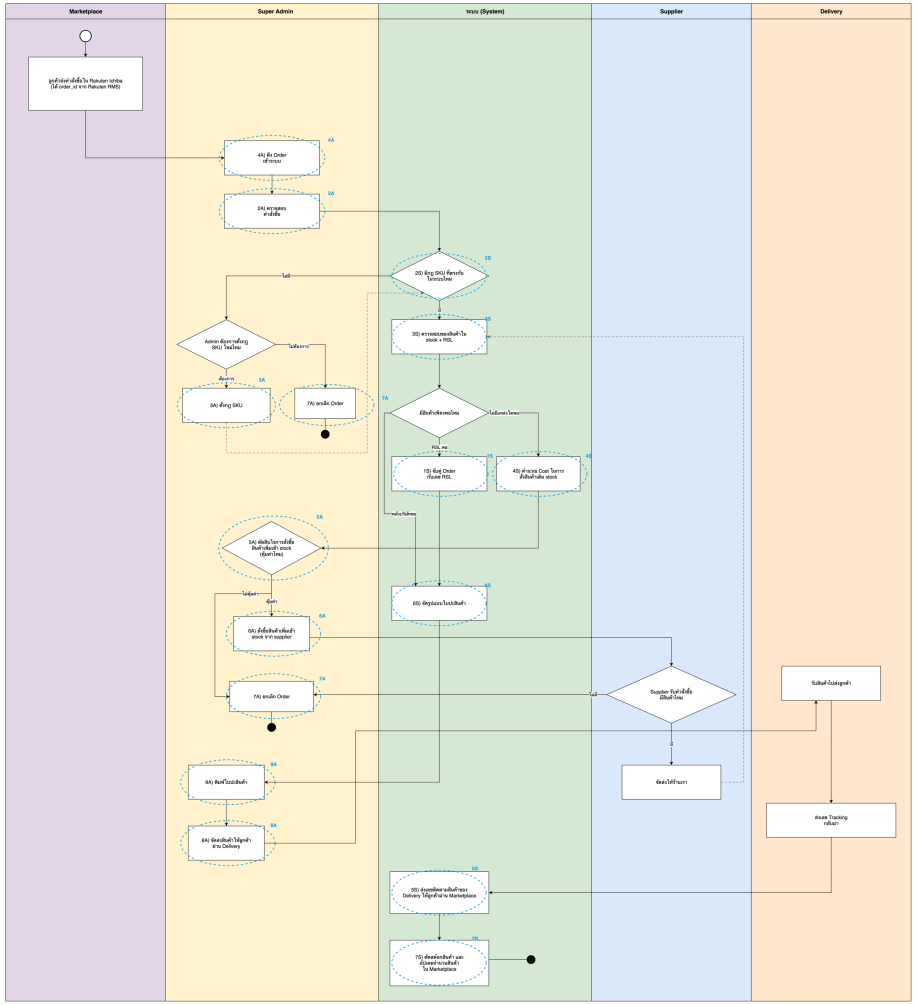
RSL พอ → 1S · คลังบริษัทพอ → 6S ทันที · ไม่มีแหล่งใดพอ → 4S/5A

To-Be: who does what

16 ขั้นตอน · A = ระบบทำ · M = คนทำ · M* = ตั้งใจให้คนตัดสินใจ

#	UC	ขั้นตอน	ผู้ทำ	แก้ปัญหา	#	UC	ขั้นตอน	ผู้ทำ	แก้ปัญหา
1	**4A**	ดึง Order เข้าระบบ	**M**	—	9	**8A**	ส่งมอบพัสดุให้ Delivery	**M**	—
2	**2A**	ตรวจสอบคำสั่งซื้อ	**M**	—	10	**5S**	ส่งเลขติดตามผ่าน Marketplace	**A**	2.3.4
3	**2S**	จับคู่กฎ SKU	**A**	—	11	**7S**	ตัดสต๊อก + อัปเดต Rakuten	**A**	2.3.4
4	**3A**	ตั้งกฎ SKU (ถ้าไม่พบ)	**M**	—	12	**4S**	คำนวณต้นทุน (Cost)	**A**	2.3.2
5	**3S**	ตรวจสต๊อก + RSL เลือกแหล่งส่ง	**A**	2.3.1	13	**5A**	ตัดสินใจสั่งซื้อเพิ่ม	**M***	2.3.2
6	**1S**	จับคู่ Order กับเลข RSL	**A**	2.3.1	14	**6A**	สั่งซื้อจาก Supplier และรับของ	**M**	—
7	**6S**	จัดรูปแบบใบปะสินค้า	**A**	2.3.3	15	**7A**	ยกเลิก Order	**M**	—
8	**9A**	พิมพ์ใบปะสินค้า	**M**	2.3.3	16	**10A**	ลบข้อมูลเก่า	**M**	ข้อมูลเก่า

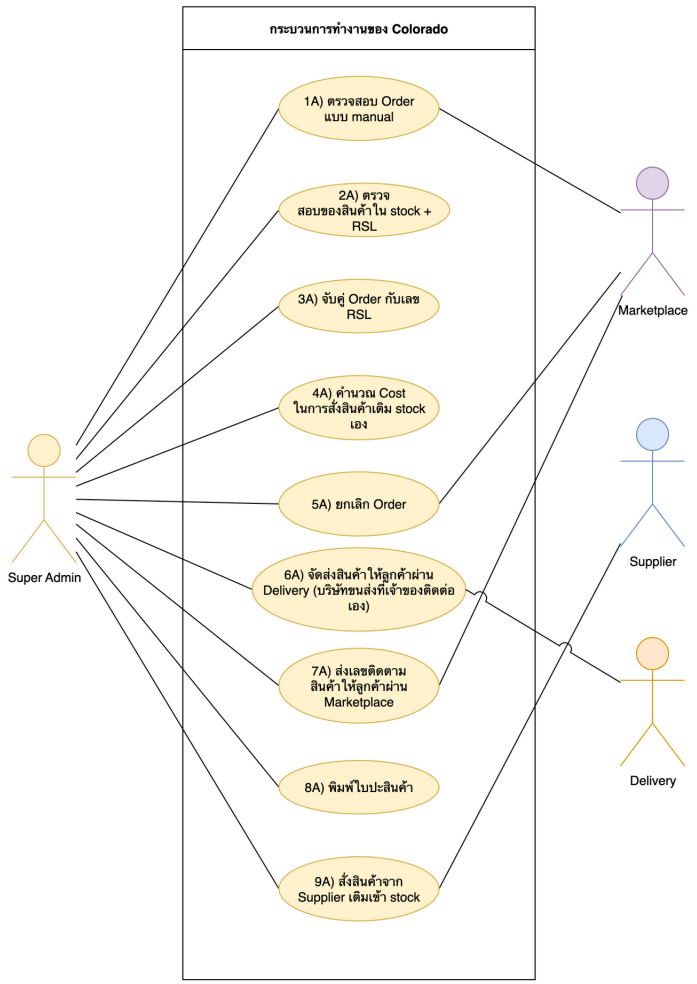
Business process to use case แต่ละกล่องใน Business Process กลายเป็น Use Case



Business Process	Use Case
ดึง Order เข้าระบบ	4A
ตรวจสอบคำสั่งซื้อ	2A
มีกฎ SKU ไหม / ตั้งกฎ SKU	2S · 3A
ตรวจสต๊อก + RSL	3S
จับคู่ Order กับเลข RSL	1S
จัดรูปแบบ / พิมพ์ใบปะ	6S · 9A
ส่งมอบ / แจ้งเลขติดตาม	8A · 5S
ตัดสต๊อก	7S
คำนวณต้นทุน (Cost)	4S
ตัดสินใจ / สั่งซื้อเพิ่ม	5A · 6A
ยกเลิก Order	7A

ไม่อยู่ในกระบวนการหลัก: 1A เข้าสู่ระบบ (ต้องทำก่อนทุกขั้น)
และ 10A ลบข้อมูลเก่า (งานดูแลระบบ)

Use case diagram: before 9 Use Case จากขั้นตอนเดิม แล้วหาสิ่งที่ขาดด้วย CRUD



CRUD ก่อนปรับปรุงชี้ว่าขาดอะไร (2.10)

1. **ไม่มีกฎ SKU**

   เพิ่ม 3A ตั้งกฎ SKU และ 2S จับคู่กฎ SKU

2. **ไม่มีขั้นตรวจ Order ก่อนเข้ากระบวนการ**

   เพิ่ม 2A ตรวจสอบคำสั่งซื้อ

3. **จัดรูปแบบกับพิมพ์ใบปะปนกัน**

   แยก 9A พิมพ์ใบปะ ออกจาก 6S

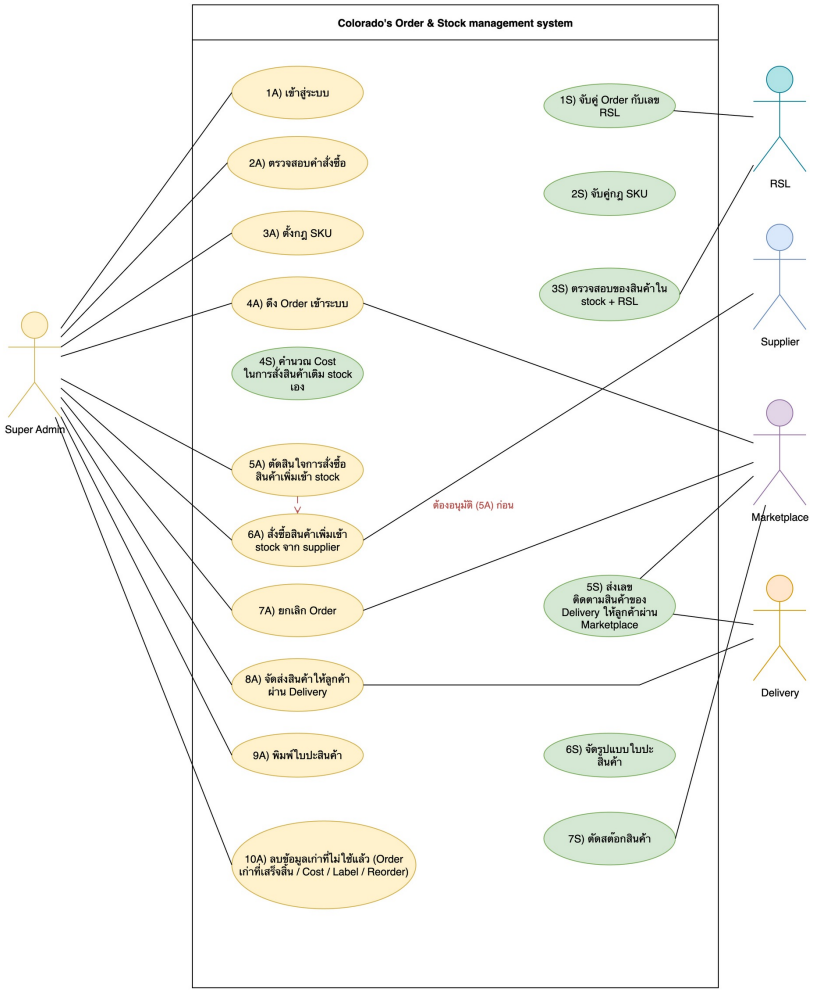
4. **งานหลังส่งมอบไม่มีเจ้าของ**

   เพิ่ม 5S แจ้งเลขติดตาม และ 7S ตัดสต๊อก

5. **ไม่มี Use Case ใดลบข้อมูลเก่า**

   เพิ่ม 10A ลบข้อมูลเก่า พร้อมเก็บประวัติการลบ

Use case diagram: reviewed 17 Use Case หลังทบทวนด้วย CRUD



เพิ่มใหม่ 7 Use Case

2A	ตรวจสอบคำสั่งซื้อ
3A	ตั้งกฎ SKU
9A	พิมพ์ใบปะสินค้า
10A	ลบข้อมูลเก่าที่ไม่ใช้แล้ว
2S	จับคู่กฎ SKU
5S	ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace
7S	ตัดสต๊อกสินค้า

Actor

Super Admin เริ่ม 10 Use Case (A) · ระบบทำ 7 Use Case (S)
ภายนอก: Marketplace, Supplier, Delivery, RSL

1S, 4S, 5S มีปุ่มให้กด แต่ระบบทำทุกขั้นเอง
จึงไม่ลากเส้นจาก Super Admin

CRUD table 17 Use Case × 17 ตาราง (entity) · แถวสีแดงคือ Use Case ที่เพิ่ม

	admins	suppliers	skus	stock	orders	order_items	costs	reorders	labels	sessions	auto_error_logs	deletion_history	cost_history	tracking_notification_logs	rakuten_stock_sync_states	stock_adjustment_history	label_print_history
1A เข้าสู่ระบบ	RU									CRU							
2A ตรวจสอบคำสั่งซื้อ			R	R	RU	RU		R	R					R	R		
3A ตั้งกฎ SKU		R	CRUD	CD	U	RU	D	R					D				
4A ดึง Order เข้าระบบ			R		CRU	C											
5A ตัดสินใจสั่งซื้อเพิ่ม			R		RU	R	R						R				
6A สั่งซื้อและรับของจาก Supplier		R	R	RU	U	R		CRU							CU	C	
7A ยกเลิก Order			R	U	RU	RU		RU	R								
8A ส่งมอบพัสดุให้ Delivery				U	RU	RU			R								
9A พิมพ์ใบปะสินค้า					RU				RU								C
10A ลบข้อมูลเก่า			R		RD	R	RD	D	RD			CR	UD				
1S จับคู่ Order กับเลข RSL			R		RU	R		R	RD								
2S จับคู่กฎ SKU			R		RU	RU		R	R		C						
3S ตรวจสต๊อก + RSL			R	RU	RU	RU					C				CU	C	
4S คำนวณต้นทุน สั่งเติมสต๊อก			R				CRU						CR				
5S แจ้งเลขติดตามผ่าน Marketplace					RU	R								CRU			
6S จัดรูปแบบใบปะสินค้า			R		RU	R		R	CR		C						
7S ตัดสต๊อกสินค้า			R	RU	RU	RU					C				CRU		

17 use cases ใครเริ่ม ทำงานตอนไหน มีกี่ query

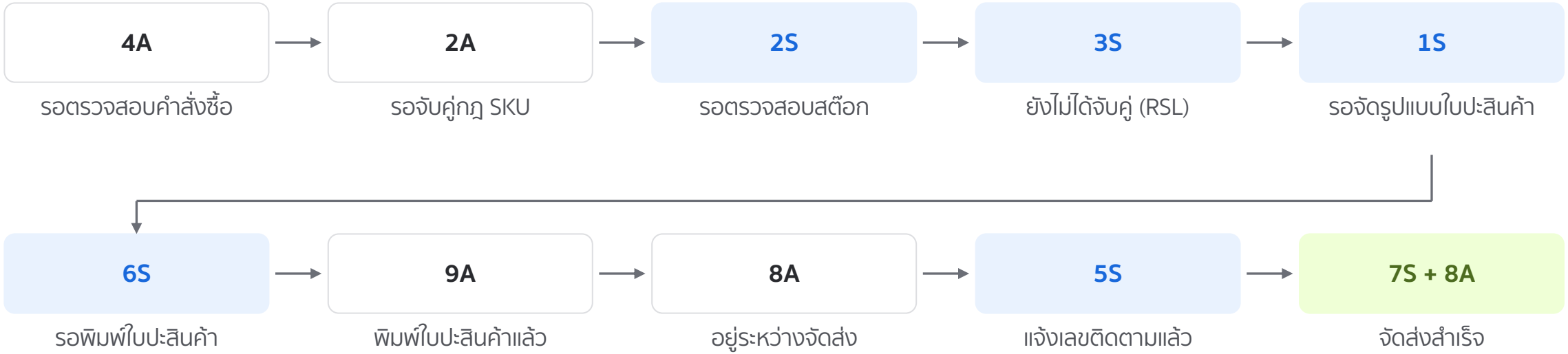
UC	Admin เริ่ม (A)	เริ่มเมื่อ	Q
1A	เข้าสู่ระบบ	Admin กรอกอีเมล/รหัสผ่าน	7
2A	ตรวจสอบคำสั่งซื้อ	Admin ยืนยัน/แก้ Order	3
3A	ตั้งกฎ SKU	Admin บันทึก/นำเข้าสินค้า	11
4A	ดึง Order เข้าระบบ	Admin กดนำเข้า Order	6
5A	ตัดสินใจสั่งซื้อเพิ่ม	Admin กดคุ้มค่า/ไม่คุ้มค่า	3
6A	สั่งซื้อจาก Supplier + รับของ	Admin ส่งใบสั่งซื้อ / รับของ	14
7A	ยกเลิก Order	Admin ใส่เหตุผลแล้วยกเลิก	5
8A	จัดส่งผ่าน Delivery	Admin ส่งมอบ / เปิดหน้าจัดส่ง	7
9A	พิมพ์ใบปะสินค้า	Admin กดพิมพ์	4
10A	ลบข้อมูลเก่าที่ไม่ใช้แล้ว	Admin เลือกข้อมูลที่จะลบ	5

UC	ระบบทำ (S)	เริ่มเมื่อ	Q
1S	จับคู่ Order กับเลข RSL	Admin กดจับคู่ RSL	4
2S	จับคู่กฎ SKU	ต่อจาก 2A / 3A	5
3S	ตรวจสต๊อก + RSL	ต่อจาก 2S / รับของ 6A	9
4S	คำนวณต้นทุน สั่งเติมสต๊อก	Admin กดคำนวณ / บันทึก 3A	7
5S	ส่งเลขติดตามผ่าน Marketplace	Admin กดส่งเลขติดตาม	4
6S	จัดรูปแบบใบปะสินค้า	ต่อจาก 1S หรือ 3S	7
7S	ตัดสต๊อกสินค้า	ต่อจาก 5S / รอบลองใหม่	12

รวม 113 query ชื่อ = Q + Use Case + ลำดับขั้น เช่น Q3S.1

ในเล่ม 2.13 ทุก Use Case มี
Description + Sequence + Collaboration เรียงติดกัน

One order, end to end Use Case ต่อกันอย่างไร และสถานะหลังแต่ละขั้น



ทางแยกที่ 3S

1 คลังบริษัทพอ (RSL ไม่พอ)

ข้าม 1S ไป 6S ทันที ใช้ใบปะแบบบริษัทส่งเอง

2 ไม่มีแหล่งใดพอ

รอ Admin ตัดสินใจสั่งซื้อ → 5A → 6A → รับของ → 3S ใหม่

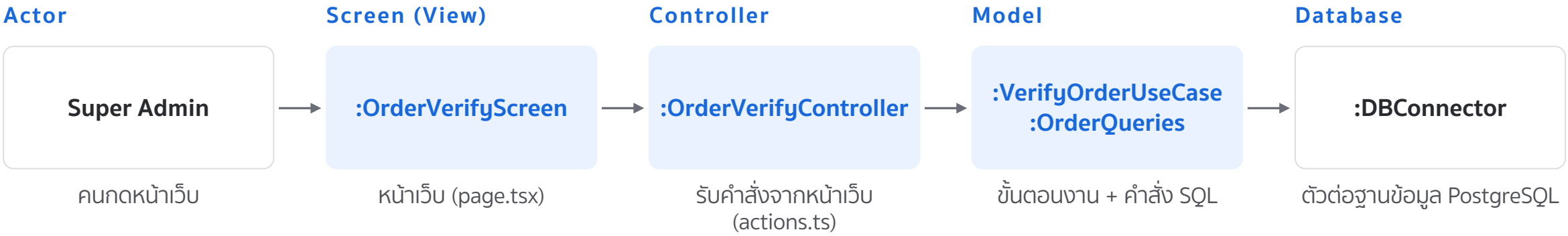
เมื่อไรจึง 'จัดส่งสำเร็จ'

- ขนส่งแจ้งว่าส่งถึงแล้ว (delivered)
- แจ้งเลขติดตามแล้ว
- ตัดสต๊อกครบทุกรายการ

ต้องครบทั้ง 3 ข้อ (ตรวจใน Q7S.7, Q5S.4, Q8A.4)

สีฟ้า = ระบบทำต่อเองทันที ในงานเดียวกับปุ่มที่ Admin กด

Sequence diagram: MVC Actor → Screen → Controller → Model → DB · ไม่มีลูกศรกลับหา Actor



ตัวอย่างจาก Sequence ของ 2A ชื่อทุกตัวตรงกับรูปในเล่ม

ลำดับ message (ย่อ) · ->> = เรียก -->> = ส่งผลกลับ

```
Admin      ->> Screen     : clickConfirm(order_id)
Screen     ->> Controller : verifyOrderAction(order_id)
Controller ->> Model      : verifyOrderUseCase(order_id)
Model      ->> DB         : query(Q2A.2)
DB        -->> Model      : return snapshot
Model     -->> Controller : return ok
Controller-->> Screen     : return result
Screen     ->> Screen     : showSuccess()
```

1. **ไม่มีลูกศรกลับหา Actor**
   Screen แสดงผลเอง (self message)
2. **Screen ไม่แตะ DB**
   ทุกคำสั่งผ่าน Controller แล้ว Model
3. **ชื่อคือบทบาท ไม่ใช่คลาสใหม่**
   Screen คือชั้น View (เกณฑ์ห้ามใช้คำว่า view)

1A Log in เข้าสู่ระบบ / ออกจากระบบ

Actor Super Admin

Pre มีบัญชีผู้ดูแล (ตาราง admins)

Post เปิด session อายุ 8 ชม.
แล้วไปหน้า Dashboard

ล็อกบัญชี 15 นาที

1. **ผิดครบ 5 ครั้ง**

   Q1A.3 ล็อกบัญชีไว้อีก 15 นาที

2. **ระหว่างล็อก**

   ไม่ตรวจรหัสผ่าน แจ้ง "กรุณารออีก N นาที"

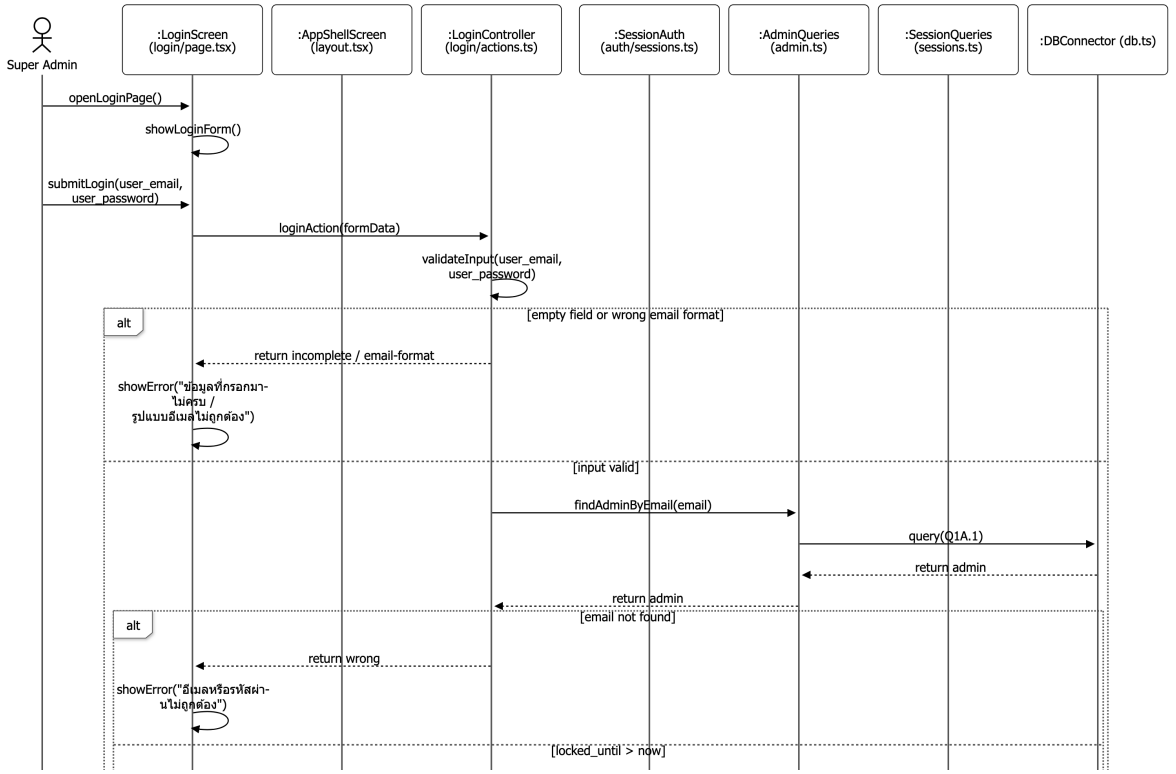
3. **ครบเวลาแล้ว**

   Q1A.2 ปลดล็อกเองก่อนตรวจรหัสผ่าน

4. **สำเร็จ**

   Q1A.4 ล้างตัวนับครั้งที่ผิด, Q1A.5 บันทึก session

Sequence 1A · ส่วนที่ 1 (เปิดหน้า Login)



2A · 2S · 3A ตรวจ Order แล้วจับคู่กฎ SKU

2A ตรวจสอบคำสั่งซื้อ

Admin · 3 query

ยืนยัน หรือแก้ SKU/จำนวน

Q2A.1 ล็อก Order กันแก้พร้อมกัน

Marketplace ตอบไม่ได้

แจ้งเตือนและคงสถานะเดิม

2S จับคู่กฎ SKU

ระบบทำ · 5 query

หา SKU จากรหัสบน Rakuten

Q2S.2 กฎอยู่ในตารางสินค้า
ไม่มีตารางกฎแยก

ไม่พบ / พบหลายกฎ

ส่งให้คนดู: 'รอดำเนินการด้วยตนเอง'

อ่านกฎไม่ได้

Q2S.5 หยิบมาทำใหม่รอบถัดไป

3A ตั้งกฎ SKU

Admin · 11 query

เพิ่ม/แก้/นำเข้าสินค้า

ใส่รหัสสินค้าบน Rakuten = ตั้งกฎ SKU

บันทึกแล้ว

Q3A.8 Order ที่ค้างกลับเข้าคิว
แล้วทำ 2S → 3S → 6S ต่อทันที

3S Check stock เลือกแหล่งส่ง: RSL ก่อน แล้วคลังบริษัท

Actor ระบบ (ทำต่อจาก 2S ทันที), RSL

Pre Order สถานะ 'รอตรวจสอบสต๊อก'

Post บันทึกแหล่งส่งที่เลือก
แล้วส่งต่อขั้นถัดไป

ขั้นตอน

1. **Q3S.1 ล็อกแถวสต๊อก**
   กันสอง Order แย่งของชิ้นเดียว
2. **Q3S.2 อ่านยอด RSL**
   ในงานเดียวกัน (transaction)
3. **Q3S.3 หักยอดที่ถูกจอง**
   Order อื่นที่ยังไม่ตัดสต๊อก
4. **Q3S.4 บันทึกแหล่งส่ง**
   ลงใน Order และรายการสินค้า

กฎเลือกแหล่ง (แต่ละแหล่งต้องพอทั้งจำนวน ไม่แบ่งส่ง)

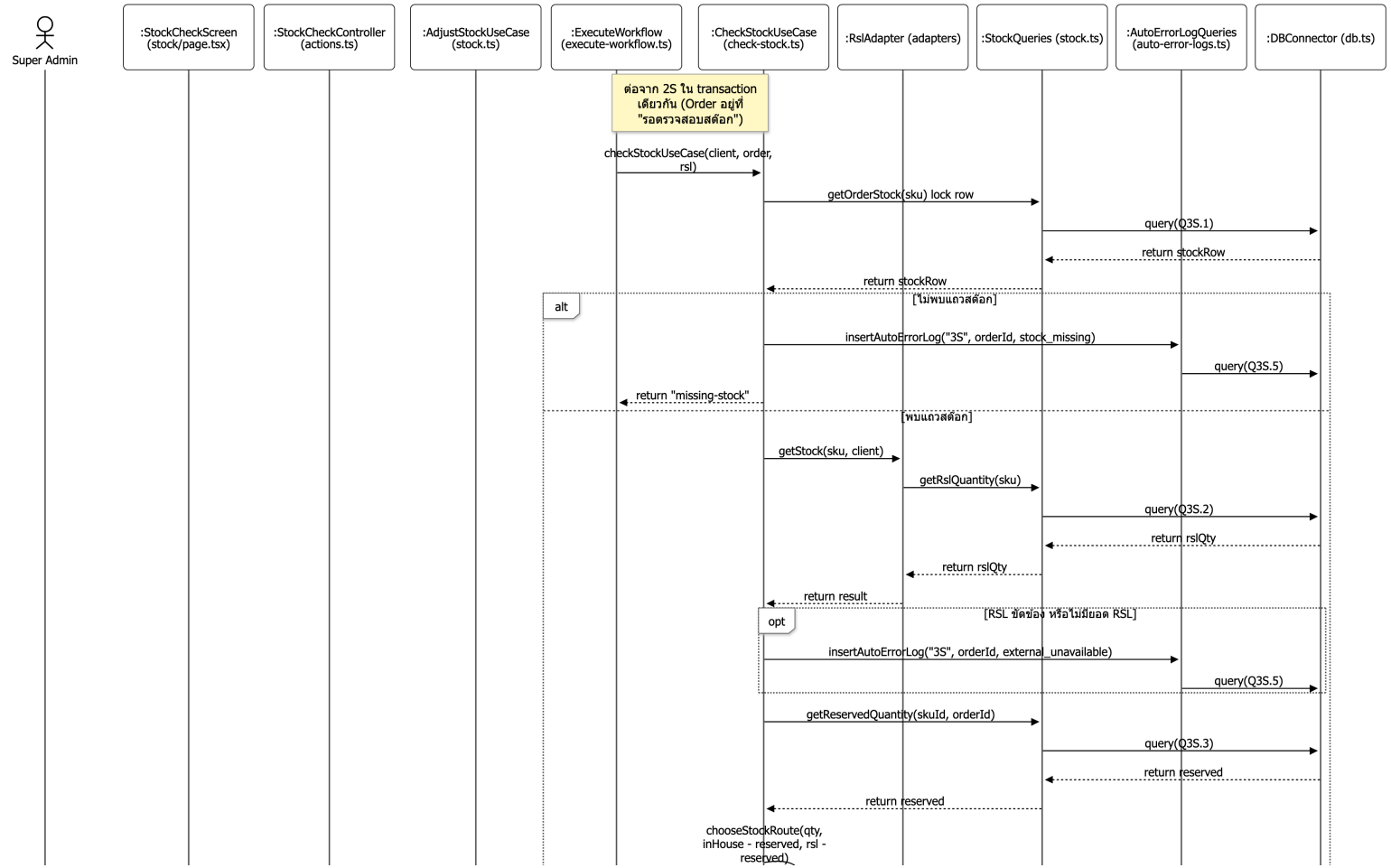
RSL ว่าง ≥ จำนวนที่สั่ง	→ ยังไม่ได้จับคู่ → 1S
RSL ไม่พอ คลังบริษัทว่าง ≥ จำนวนที่สั่ง	→ รอจัดรูปแบบใบปะสินค้า → 6S ทันที
ไม่มีแหล่งใดพอ	→ รอ Admin ตัดสินใจสั่งซื้อ → 5A

```
ว่าง = ยอดในคลัง − ยอดที่ Order อื่นจองไว้ (Q3S.3)
RSL ล่ม → ใช้เฉพาะคลังบริษัท และ Q3S.5 บันทึกข้อผิดพลาด
```

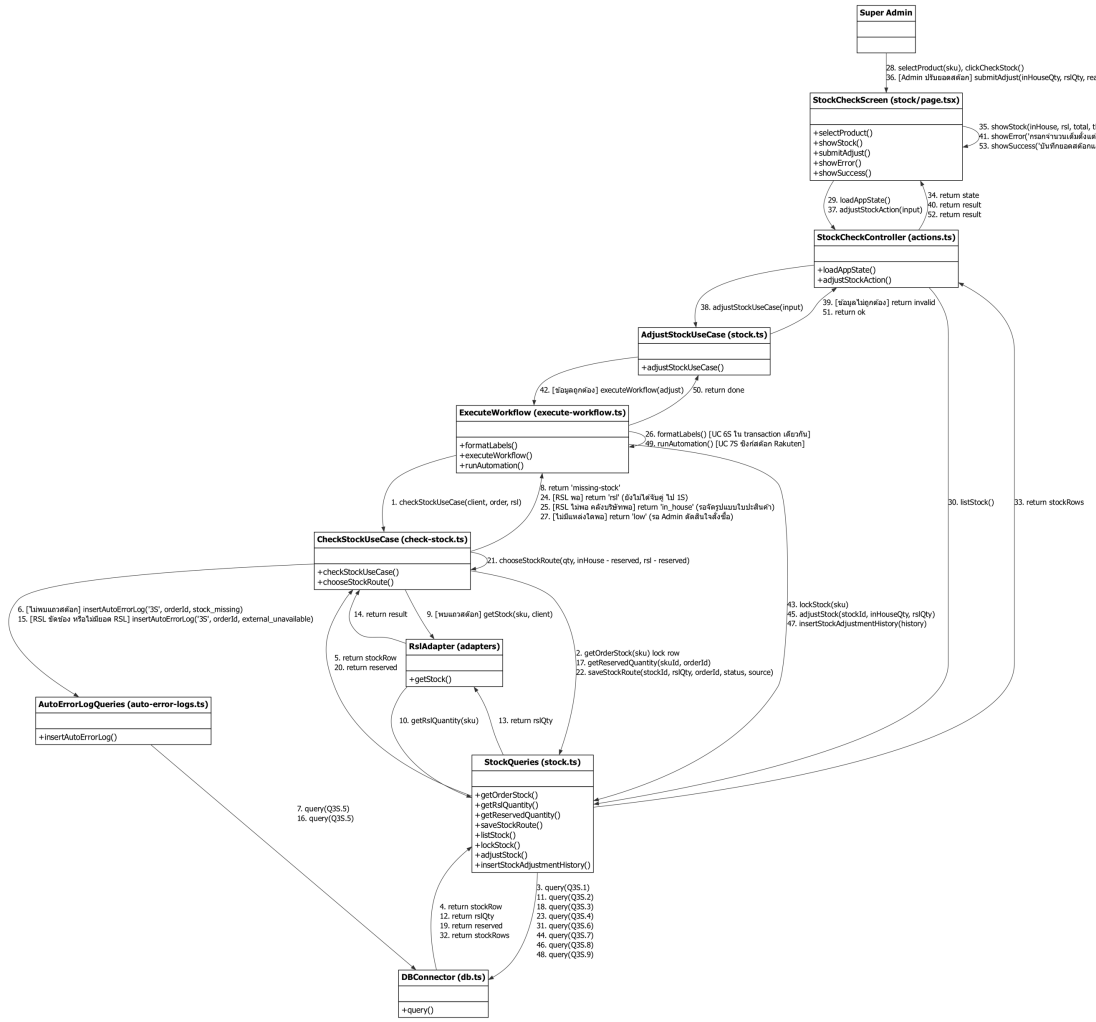
Admin ปรับยอดสต๊อกเองได้ที่หน้าตรวจสต๊อก
(Q3S.6–Q3S.9 เก็บประวัติการปรับทุกครั้ง)

3S Sequence diagram ส่วนอัตโนมัติ: ล็อก → หักยอดจอง → เลือก RSL / คลังบริษัท / ไม่พอ

Sequence 3S · ส่วน 1/3 ในเล่ม



3S Collaboration diagram object และ message ชุดเดียวกับ Sequence



1 แปลงจาก Sequence

object ชุดเดียวกัน message ครบทุกเส้น

2 เลขลำดับแทนแกนเวลา

1, 2, 3 ... ตามลำดับใน Sequence

3 [route = ...] กำกับทางเลือก

rsl = RSL พอ
in_house = คลังบริษัทพอ
low = ไม่มีแหล่งใดพอ

4 DBConnector อยู่ล่างสุด

Queries ทุกตัวเข้าฐานข้อมูลผ่านตัวนี้

1S · 6S · 9A Labels Auto label: จับคู่ → จัดรูปแบบ → พิมพ์

1 1S จับคู่ Order กับเลข RSL (4 query)

รับเฉพาะ Order ที่ส่งจาก RSL
Order ที่บริษัทส่งเองถูกปฏิเสธ

2 6S จัดรูปแบบใบปะ (7 query)

เลือกแบบใบปะตามแหล่งส่งและวิธีส่ง
Q6S.4 สร้างใบปะ สถานะ ready (พร้อมพิมพ์)

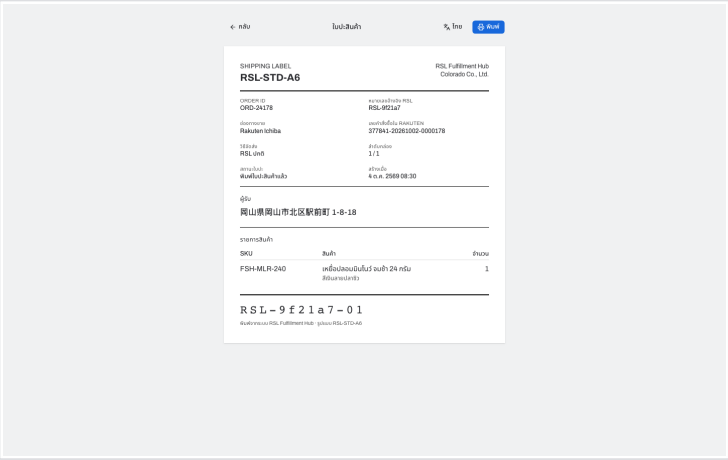
3 9A พิมพ์ใบปะ (4 query)

Admin กดพิมพ์
Q9A.3 บันทึกประวัติทุกครั้ง รวมพิมพ์ซ้ำ

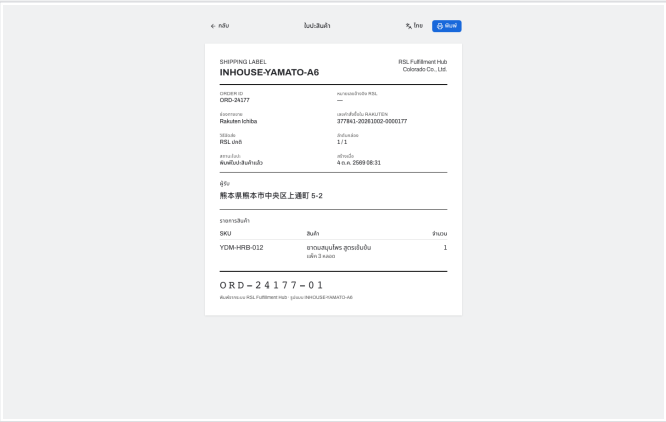
แบบใบปะ (template)

แหล่งส่ง	แบบใบปะ
RSL ส่งแบบปกติ	**RSL-STD-A6**
RSL ส่งของใหญ่	**RSL-LARGE-A5**
บริษัทส่งเอง	**INHOUSE-YAMATO-A6**

ใบปะ RSL



ใบปะ บริษัทส่งเอง (ไม่มีเลข RSL)



4S Unit cost สูตรเดียวทุกครั้ง พร้อมประวัติ

สูตรต้นทุนต่อหน่วย (2.22)

```
Unit Cost = ((purchase_price × exchange_rate) + intl_freight + duty_fee) / order_qty
          + marketplace_fee + domestic_shipping + rsl_charge
```

ต้นทุนต่อหน่วย = ((ราคาซื้อ × อัตราแลกเปลี่ยน) + ค่าขนส่งระหว่างประเทศ + ภาษีนำเข้า) ÷ จำนวนที่สั่ง
+ ค่าธรรมเนียม Marketplace + ค่าส่งในประเทศ + ค่า RSL

1 เริ่มเมื่อ

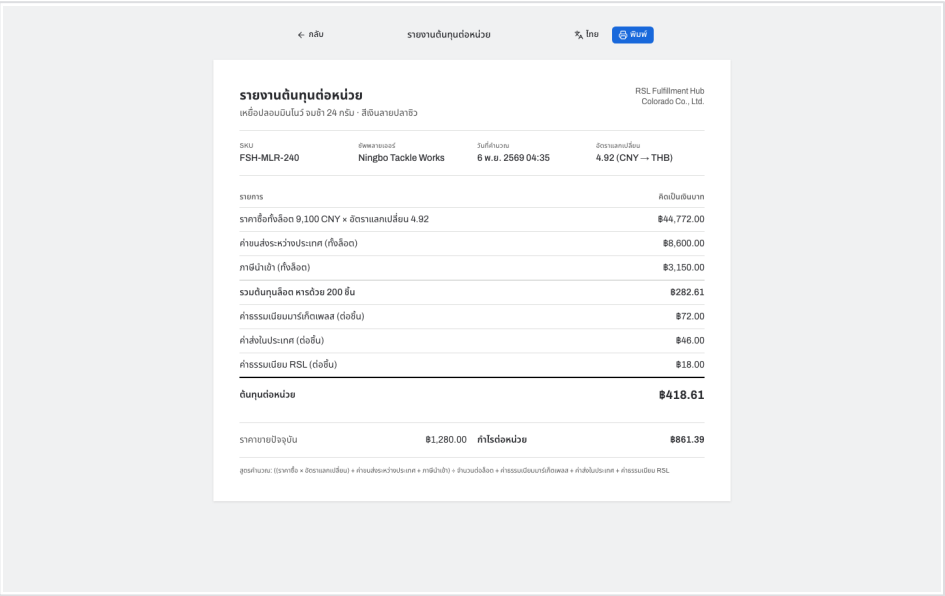
Admin กดคำนวณที่หน้าต้นทุน
หรือบันทึกสินค้าพร้อมต้นทุนใน 3A

2 ข้อมูลไม่ครบ

"กรุณากรอกข้อมูลต้นทุนให้ครบถ้วนและถูกต้อง"

3 บันทึก

Q4S.5 เก็บค่าล่าสุดไว้ใช้ใน 5A
Q4S.6 เก็บประวัติทุกครั้ง (รวม 7 query)



5A Decide reorder ระบบเตรียมข้อมูล คนกดตัดสินใจ

Actor Super Admin

Pre Order สถานะ
'รอ Admin ตัดสินใจสั่งซื้อ'
(3S พบว่าสต๊อกไม่พอ)

Post คุ้มค่า → รอสั่งซื้อจาก Supplier (6A)
ไม่คุ้มค่า → รอยกเลิก Order (7A)

กรณีผิดพลาด

- จำนวนไม่ใช่จำนวนเต็มบวก → แจ้งให้แก้
- ไม่มีราคาขาย → ลิงก์ไปตั้งราคาที่ 3A
- ถูกตัดสินใจไปแล้ว → ไม่บันทึกซ้ำ

1 Q5A.1 ล็อก Order แล้วดึงข้อมูล

ราคาขาย + ต้นทุนล่าสุดจาก 4S + เหตุผลที่ Order ค้าง

2 Admin กดเลือกเอง

ปุ่ม "คุ้มค่า - สั่งซื้อเพิ่ม" พร้อมจำนวน หรือ "ไม่คุ้มค่า - ยกเลิก"

Q5A.2 บันทึกผลการตัดสินใจ

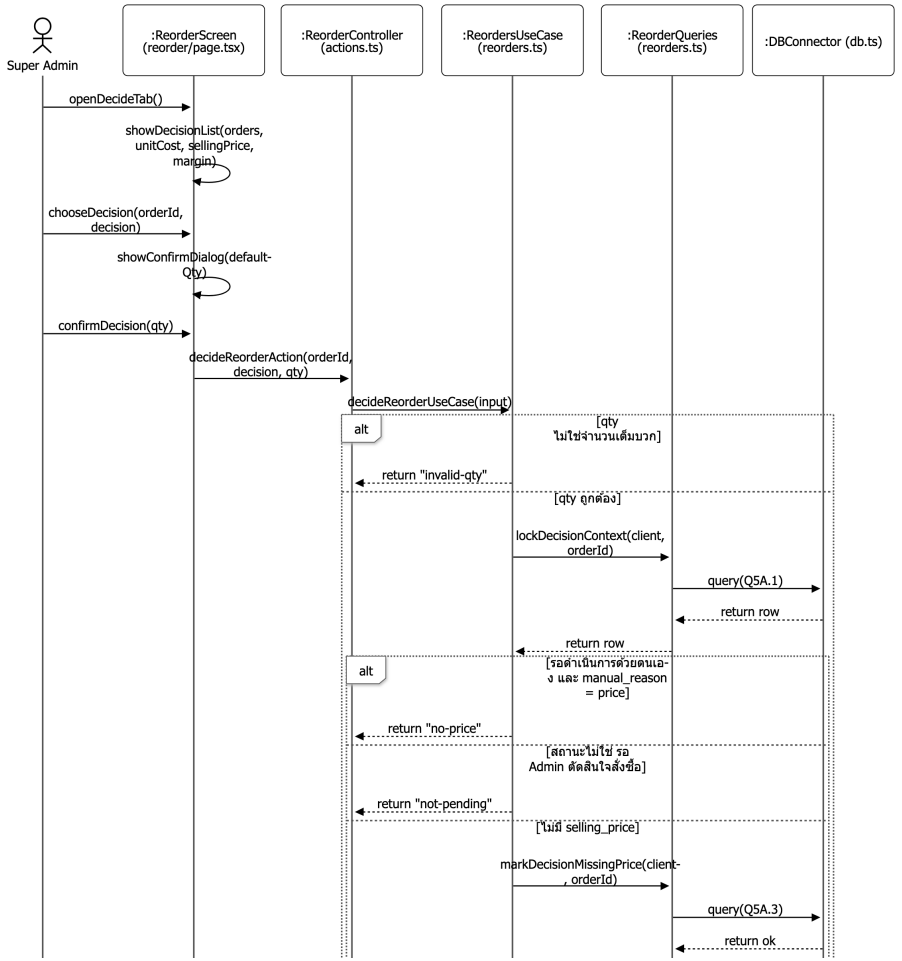
- ผลการตัดสินใจ: คุ้มค่า / ไม่คุ้มค่า
- จำนวนที่จะสั่ง (เฉพาะเมื่อคุ้มค่า)
- สถานะใหม่ของ Order และเวลาที่แก้

Q5A.3 ไม่มีราคาขาย

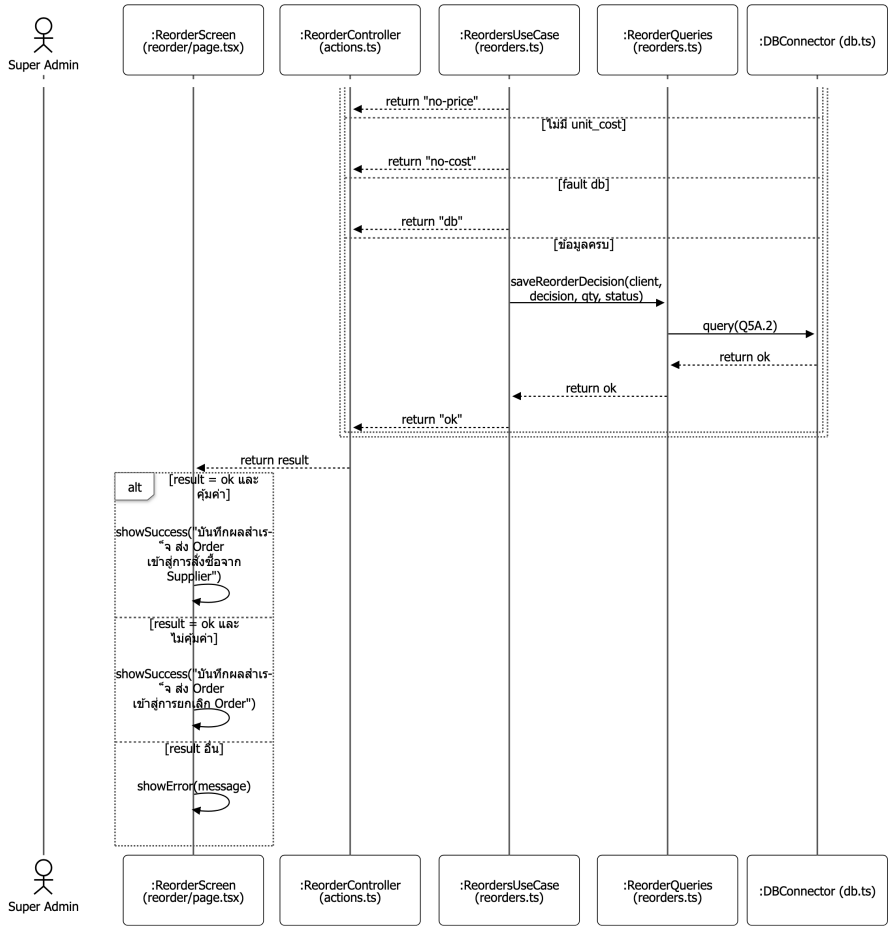
Order เป็น 'รอดำเนินการด้วยตนเอง' พร้อมบันทึกเหตุผลว่าไม่มีราคา

5A Sequence diagram Screen → Controller → Model → DBConnector

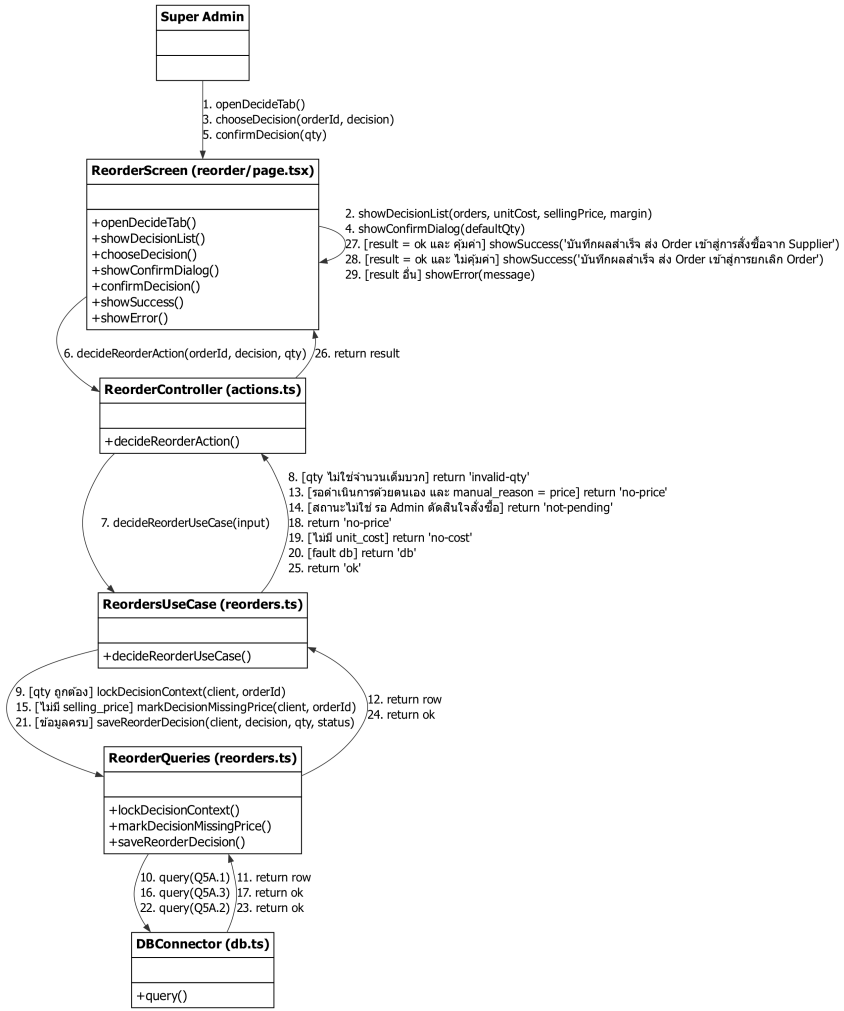
Sequence 5A · ส่วน 1/2 ในเล่ม



Sequence 5A · ส่วน 2/2 ในเล่ม



5A Collaboration diagram object ชุดเดียวกับ Sequence · เลขลำดับแทนแกนเวลา



1 :ReorderScreen

Screen · หน้าตัดสินใจสั่งซื้อ

2 :ReorderController

Controller · รับคำสั่งจากหน้าจอ (actions.ts)

3 :ReordersUseCase

Model · ตัดสินใจและบันทึก Q5A.1–Q5A.3

4 :DBConnector

Database · ตัวต่อฐานข้อมูล (db.ts)

6A Reorder and receive ส่งใบสั่งซื้อ แล้วรับของเข้าคลังบริษัท

ส่งใบสั่งซื้อ (Q6A.1–Q6A.9)

1. **Q6A.6 ขอเลขใบสั่งซื้อ**
   ระบบออกเลขถัดไปให้ เช่น PO-0092
2. **Q6A.7 สร้างใบสั่งซื้อ**
   สถานะ 'รอส่งคำสั่งซื้อ'
3. **Q6A.8 ผูกทุก Order ของ SKU นี้**
   ใบสั่งซื้อเดียวครอบคลุมหลาย Order
4. **ส่งให้ Supplier (ผ่านตัวเชื่อม)**
   สำเร็จ → Q6A.9 'สั่งซื้อแล้ว'
   พร้อมวันที่คาดว่าจะได้รับ

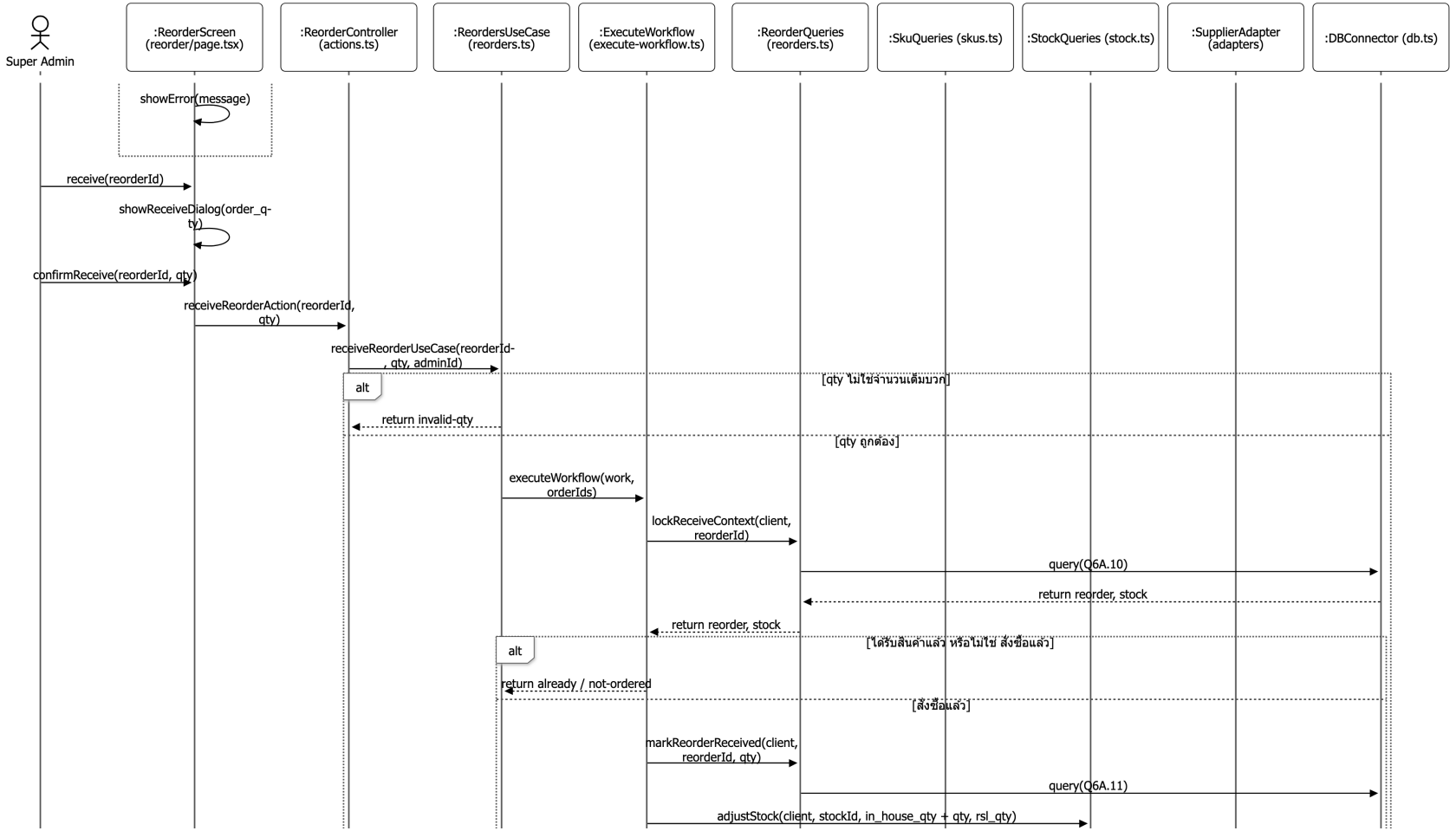
รับสินค้า (Q6A.10–Q6A.14)

1. **Q6A.10 ล็อกใบสั่งซื้อ + สต๊อก**
   กันกดรับของพร้อมกันสองครั้ง
2. **Q6A.11 'ได้รับสินค้าแล้ว'**
   เงื่อนไขสถานะทำให้รับซ้ำไม่ได้
3. **Q6A.12–13 เพิ่มสต๊อกคลังบริษัท**
   พร้อมบันทึกประวัติการปรับสต๊อก
4. **Q6A.14 Order ที่ผูกไว้ → 'รอตรวจสอบสต๊อก'**
   แล้ว 3S, 6S ทำต่อทันที

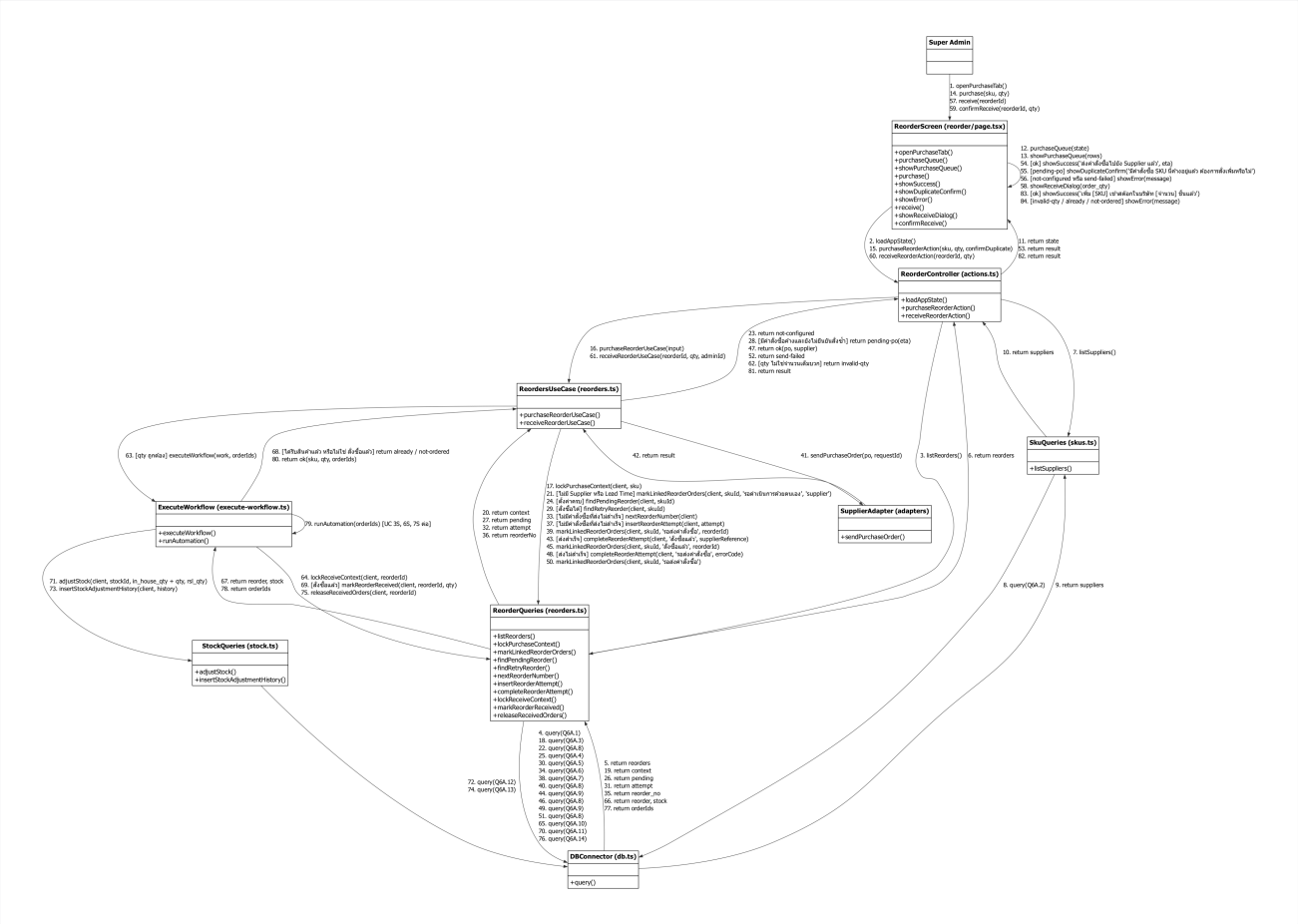
ทดสอบจริง รับ PO-0091 จำนวน 60 ชิ้น → "เพิ่ม OUT-CHR-003 เข้าสต๊อกในบริษัท 60 ชิ้นแล้ว" และ Order ที่ผูกไว้ 2 รายการกลับไปตรวจสต๊อก

6A Sequence diagram ช่วงรับสินค้า: Screen → Controller → UseCase → ExecuteWorkflow

Sequence 6A · ส่วน 4/5 ในเล่ม



6A Collaboration diagram message มีเลขลำดับ ครบทั้งส่งและรับ



1 **ReorderScreen → ReorderController**

Screen → Controller ปุ่มสั่งซื้อและรับสินค้าอยู่หน้าเดียวกัน

2 **SupplierAdapter**

ตัวเชื่อมไป Supplier ตอนนี้เป็นตัวจำลอง

3 **ExecuteWorkflow**

รับของแล้วเรียก 3S, 6S ต่อ

4 **Q6A.1–Q6A.14**

ทุกเลขตรงกับ Description

8A · 5S · 7S Ship and deduct ส่งมอบ → แจ้งเลขติดตาม → ตัดสต๊อก

8A ส่งมอบพัสดุ

Admin | 7 query

Q8A.2 บันทึกเลขติดตาม

มีเลข → 'อยู่ระหว่างจัดส่ง'
ไม่มีเลข → 'รอส่งมอบ'

เปิดหน้าจัดส่ง

ขนส่งแจ้งส่งถึง → 7S
แล้ว Q8A.5 ปิดงาน

5S แจ้งเลขติดตาม

ระบบทำ | 4 query

Q5S.2 กันส่งซ้ำ

เช็กประวัติการแจ้งก่อนส่ง

ส่งผ่านตัวเชื่อม Rakuten

สำเร็จ → 'แจ้งเลขติดตามแล้ว'
แล้ว 7S ทันที

7S ตัดสต๊อก

ระบบทำ | 12 query

Q7S.2 ล็อกรายการ + สต๊อก

ตัดเฉพาะรายการที่ยังไม่ตัด

Q7S.6 คิวอัปเดต Rakuten

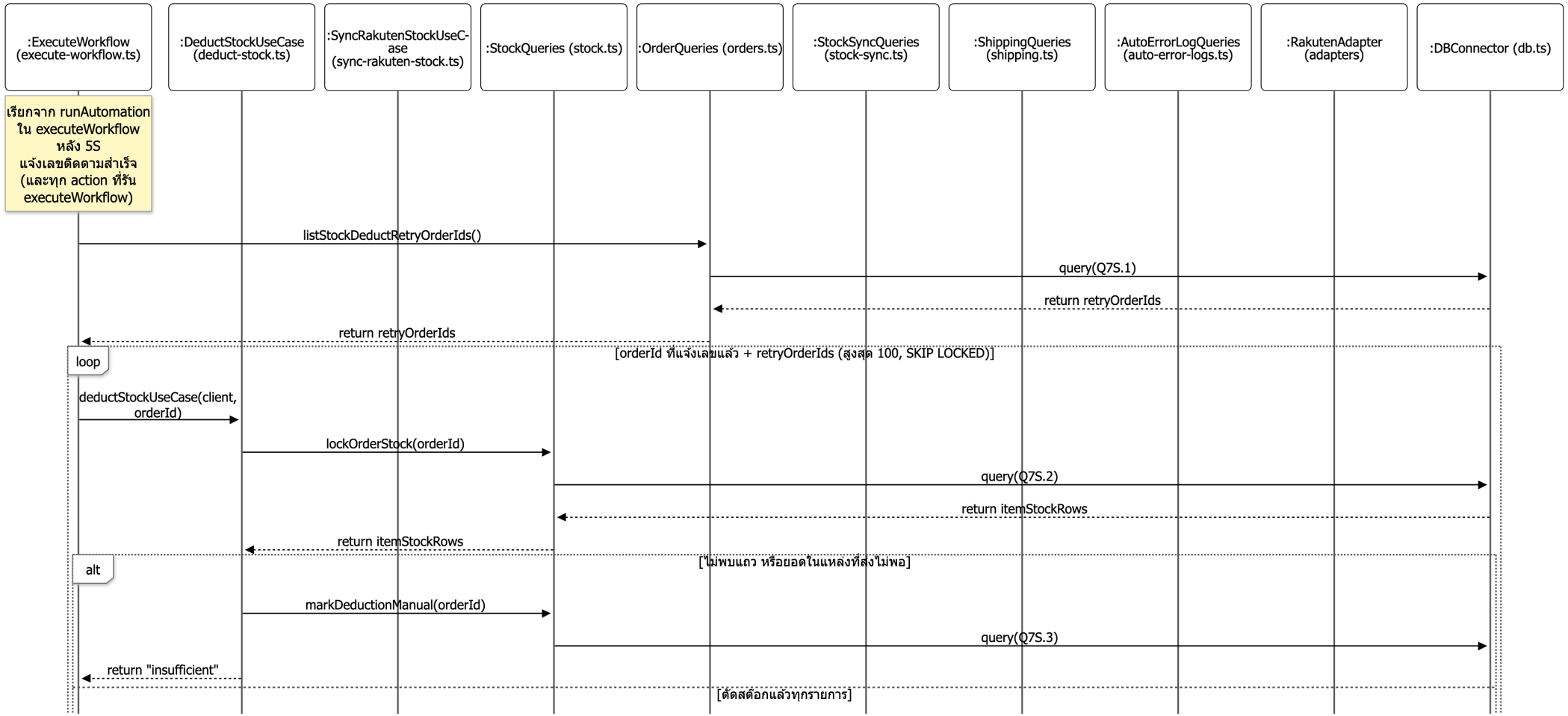
Rakuten ล่ม: ตัดในระบบไว้ก่อน
แล้วลองใหม่รอบถัดไป

'จัดส่งสำเร็จ' ต้องครบ 3 เงื่อนไข

ขนส่งแจ้งส่งถึง (delivered) + แจ้งเลขติดตามแล้ว + ตัดสต๊อกครบทุกรายการ
7S ไม่ตั้งสถานะนี้เอง ถ้าขนส่งยังไม่แจ้งส่งถึง · ทุกครั้งที่ Admin กดปุ่ม Q7S.1 หยิบ Order ที่ค้างมาตัดใหม่

7S Sequence diagram รอบลองใหม่ (Q7S.1) → ตัดสต๊อก → ส่งยอดให้ Rakuten

Sequence 7S · ส่วน 1/3 ในเล่ม



4A · 7A · 10A นำเข้า ยกเลิก และลบข้อมูลเก่า

4A ดึง Order เข้าระบบ

6 query

Admin กดนำเข้าที่ Dashboard

ตัวเชื่อม Rakuten ดึง Order ใหม่

Q4A.5 บันทึก Order + รายการ

ข้าม Order ซ้ำ
แจ้ง Order ไม่ครบ / SKU ยังไม่ลงทะเบียน

Q4A.6 ลูกค้าขอยกเลิก

ทำเครื่องหมายว่าลูกค้าขอยกเลิก

7A ยกเลิก Order

5 query

ต้องใส่เหตุผล

"กรุณาระบุเหตุผลการยกเลิก"

Q7A.3 คืนสต๊อกตามคลังที่ตัด

ถ้าตัดไปแล้ว

ส่งมอบแล้ว → 'รอดำเนินการพิเศษ'

Q7A.4–5 ยกเลิกใบสั่งซื้อที่ยังเปิด
Order อื่นที่ผูกไว้กลับไป 5A

10A ลบข้อมูลเก่า

5 query

เฉพาะข้อมูลที่ปิดแล้ว

Order 'จัดส่งสำเร็จ' / 'ยกเลิกแล้ว'
ใบสั่งซื้อ (PO) ที่จบแล้ว

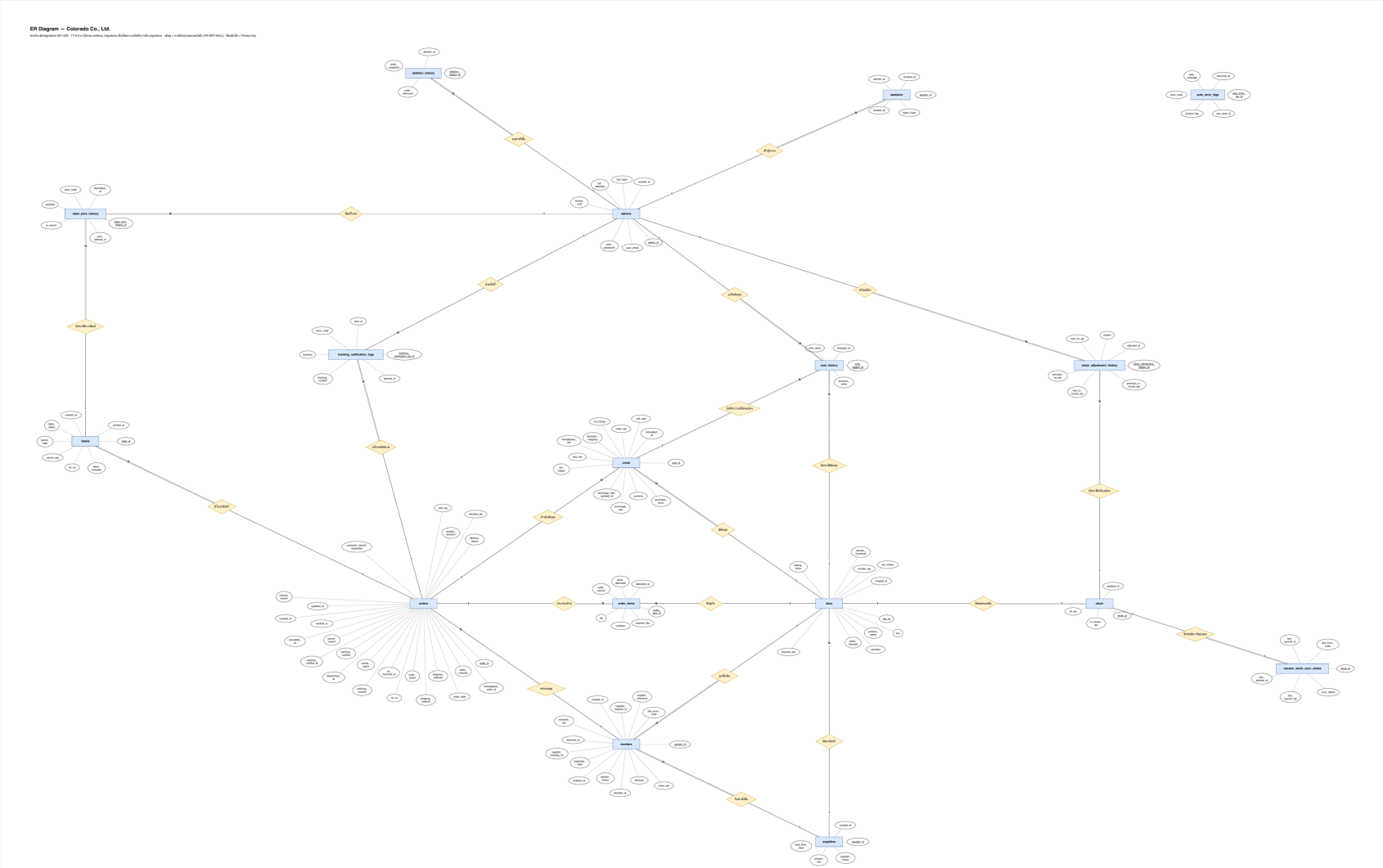
Q10A.1 เก็บสำเนาก่อนลบ

Order + รายการ + ใบปะ
ลงประวัติการลบ

ลบทั้งชุด หรือไม่ลบเลย

ไม่ลบใบสั่งซื้อที่ Order
ยังเปิดอ้างอยู่

ER diagram 17 entity · 22 relationship · ไม่มี M:N (หลายต่อหลาย)



ER แบบ Chen = ฐานข้อมูลจริง

ตรงกับไฟล์สร้างตาราง 001–008
ไม่นับ schema_migrations
(ตารางจดเวอร์ชันฐานข้อมูล)

ทุกเส้น 1:1 หรือ 1:N

เส้นคู่ = ทุกแถวต้องมีคู่
(total participation)

ใบสั่งซื้อ 1 ใบ : หลาย Order

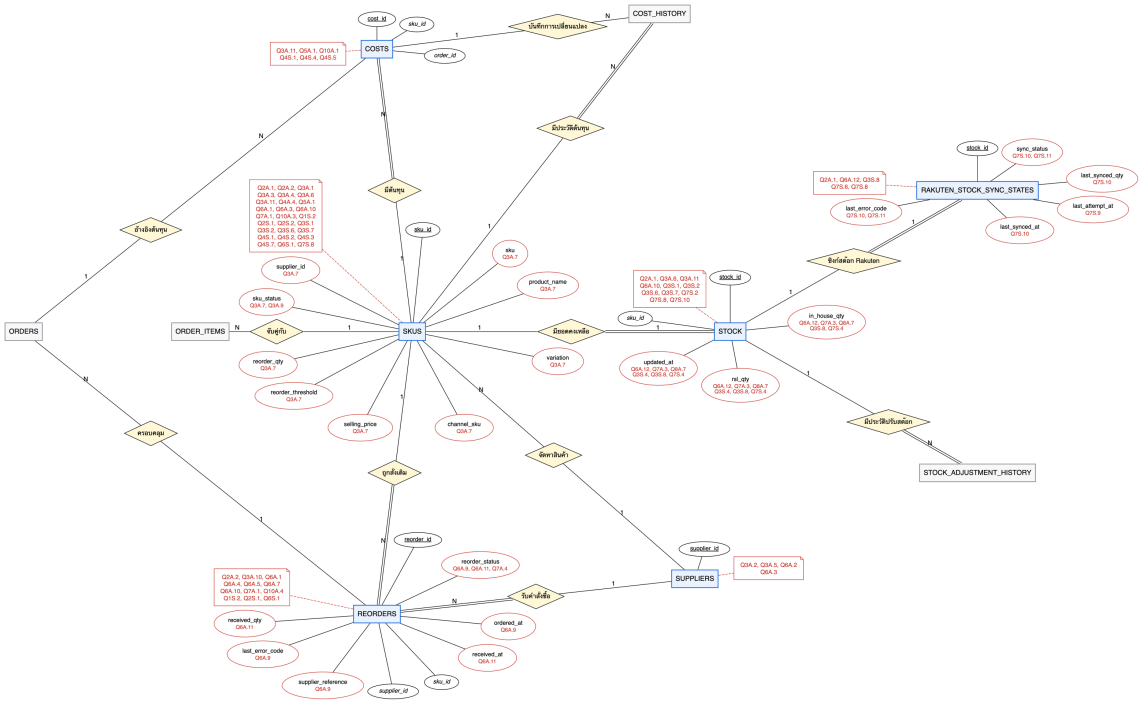
เก็บเลขใบสั่งซื้อไว้ใน Order
ไม่ต้องมีตารางเชื่อม

Order 1 : หลายรายการสินค้า

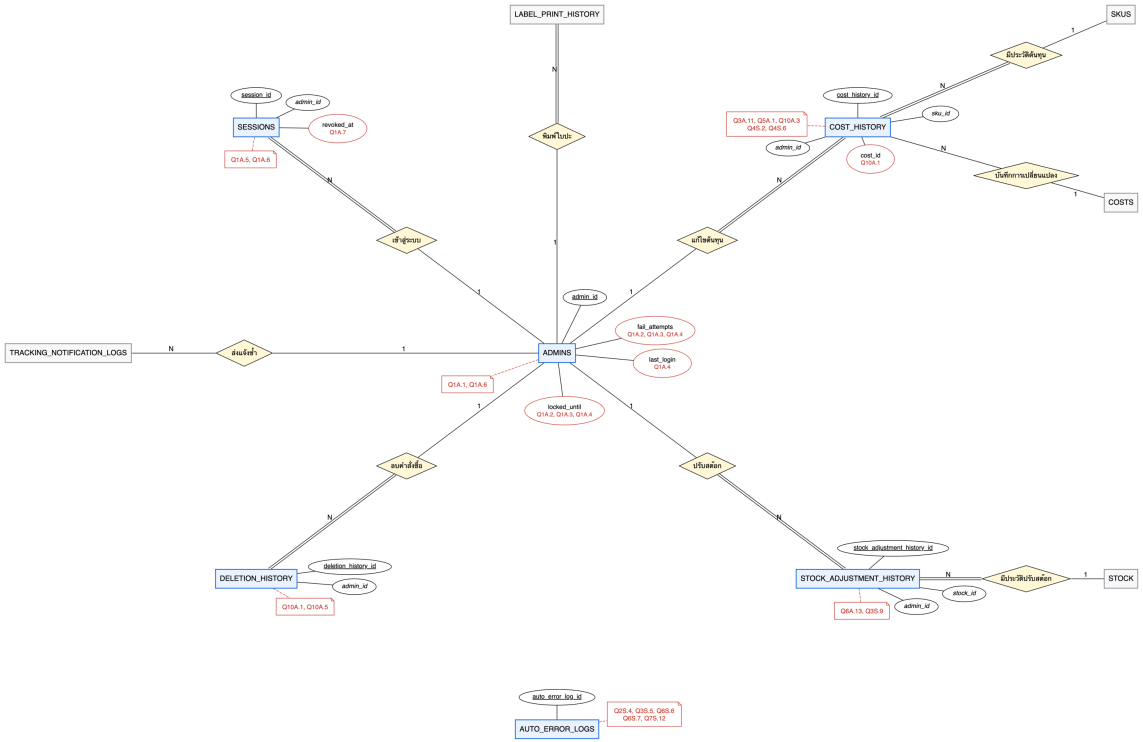
แยกรายการไว้ใน order_items
จึงไม่เกิด M:N กับ skus

ER diagram with queries ป้ายสีแดง = query ที่กระทำกับ entity (113 query)

กลุ่มสินค้า สต๊อก และการสั่งซื้อ



กลุ่มจัดส่งและใบปะ



Tables and queries 17 ตาราง · 113 query

17 ตารางในฐานข้อมูล (2.17)

หลัก	admins, suppliers, skus, stock, costs
Order	orders, order_items, reorders, labels
ประวัติ	cost_history, stock_adjustment_history, label_print_history, tracking_notification_logs, deletion_history
ระบบ	sessions, auto_error_logs, rakuten_stock_sync_states

ในเล่มแต่ละตารางมี

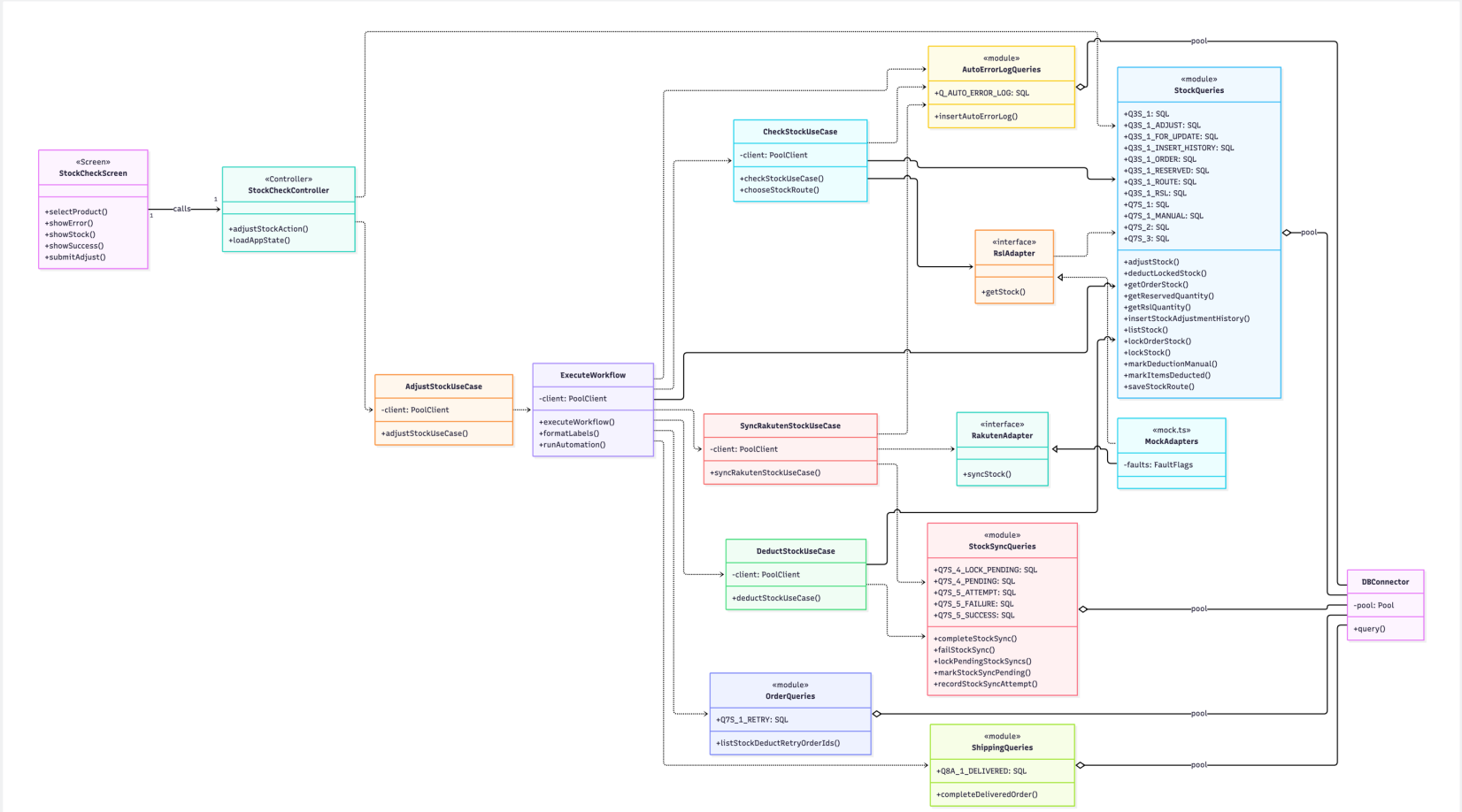
- attribute, คำอธิบาย, ชนิดข้อมูล
- key (PK/FK), NOT NULL, CHECK
- ตัวอย่างข้อมูล 5 แถว
- ยกเว้น admins มี 2 แถว (มี Super Admin คนเดียว)

ตาราง Query (2.16) ตัวอย่าง

Query	UC	SQL (ย่อ)
Q3S.1	3S	`SELECT stock.stock_id, stock.sku_id, stock.in_house_qty, stock.rsl_qty…`
Q3S.3	3S	`SELECT COALESCE(SUM(item.qty) FILTER (WHERE item.fulfill_source = 'rsl…`
Q6A.14	6A	`UPDATE orders SET order_status = 'รอตรวจสอบสต๊อก', manual_reason = NUL…`
Q7S.5	7S	`UPDATE order_items SET stock_deducted = TRUE, deducted_at = CURRENT_TI…`
Q10A.5	10A	`SELECT order_reference, admin_id, deleted_at FROM deletion_history ORD…`

SQL ทุกข้อคัดลอกจากโค้ดของเว็บตรงๆ
ในเล่มบอกชื่อไฟล์ที่ใช้คำสั่งนั้นด้วย

Class diagram 15 รูป ได้จาก object ทุกตัวใน Sequence



รูปนี้คือกลุ่ม 3S + 7S

12 รูปตามกลุ่มงาน
+ 3 รูปของคลาสข้อมูล (Entity)

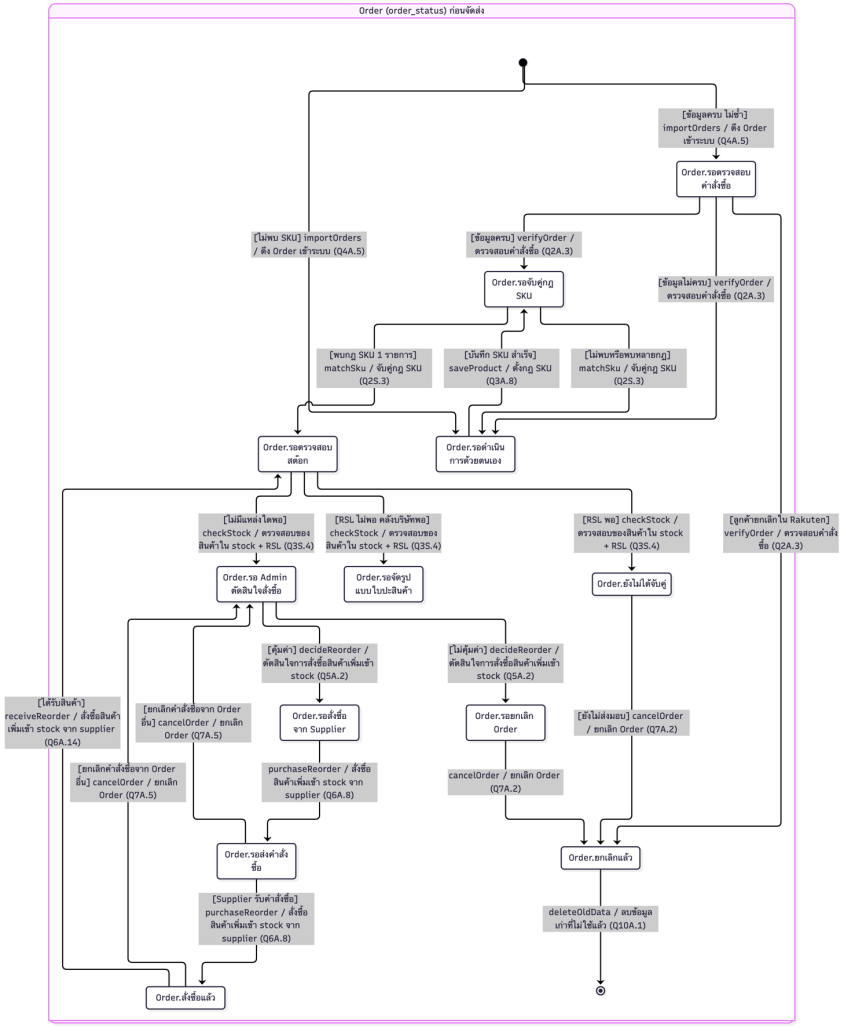
ชั้นของคลาส

View = Screen (หน้าเว็บ)
Controller = รับคำสั่งจากหน้าเว็บ
Model = UseCase + ExecuteWorkflow
Queries = เก็บคำสั่ง SQL
DBConnector = ตัวต่อฐานข้อมูล
Adapter = ตัวเชื่อมระบบภายนอก

ความสัมพันธ์

association = ถือ object อีกตัวไว้ใช้
dependency = เรียกใช้ชั่วคราว
aggregation = ประกอบด้วยส่วนย่อย
realization = ทำตาม interface

State diagram: Order (1/3) สถานะ Order · อ่านเส้น: [เงื่อนไข] เหตุการณ์ / Use Case (Query)



ตัวอย่างเส้น

```
[RSL ไม่พอ คลังบริษัทพอ] / 3S (Q3S.4)
รอตรวจสอบสต๊อก → รอจัดรูปแบบใบปะสินค้า
```

1 **22 สถานะ ตรงกับโค้ดของเว็บ**

ทุกเส้นคือคำสั่งแก้สถานะใน query นั้น

2 **3S ออกได้ 3 ทาง**

RSL / คลังบริษัท / รอ Admin ตัดสินใจ

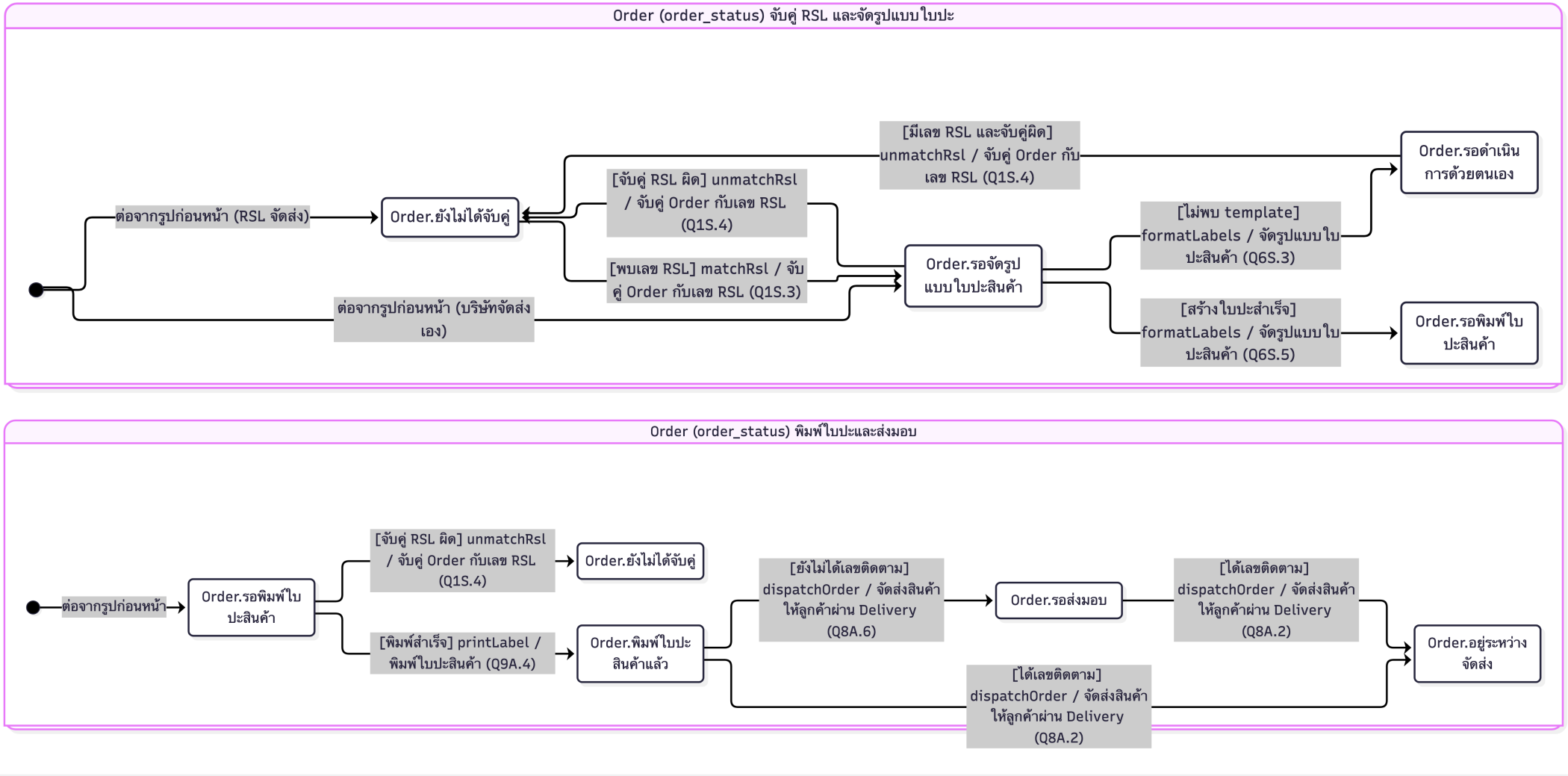
3 **'สั่งซื้อแล้ว' ออกได้ 2 ทาง**

รับของ 6A (Q6A.14) หรือยกเลิกใบสั่งซื้อ 7A (Q7A.5)

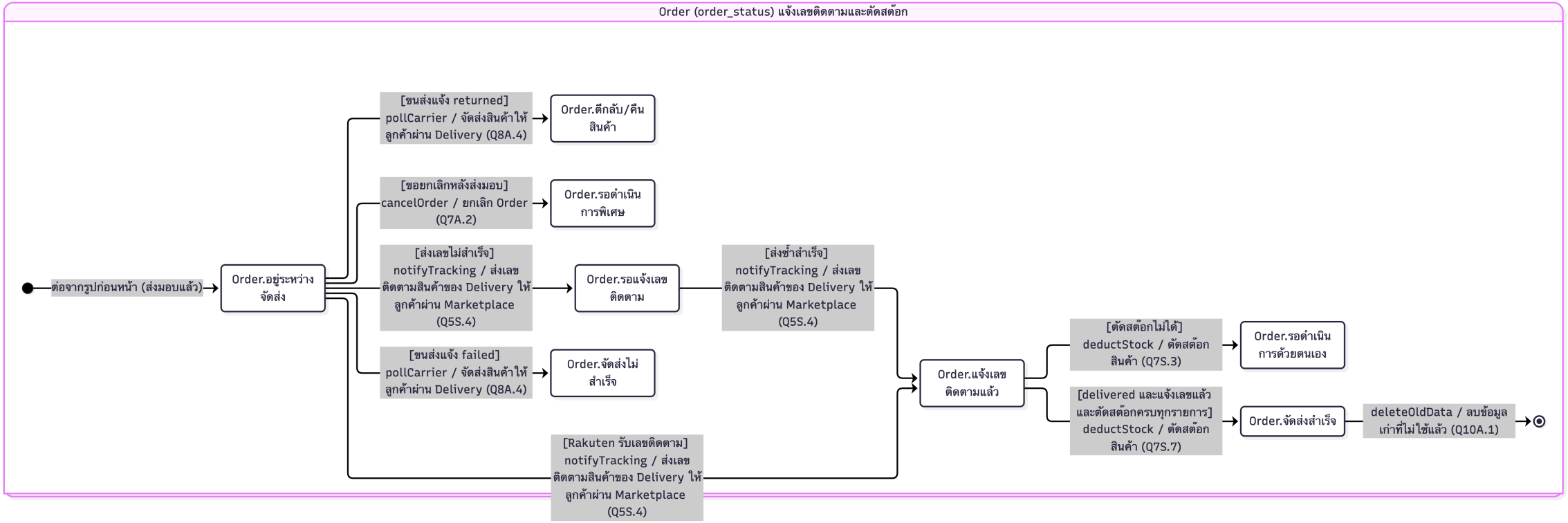
4 **รอดำเนินการด้วยตนเอง**

บันทึกสาเหตุไว้ เช่น ไม่พบกฎ SKU, ไม่มีราคา

State diagram: Order (2/3) บน: จับคู่ RSL · จัดรูปแบบใบปะ ล่าง: 9A พิมพ์ใบปะ · 8A ส่งมอบ



State diagram: Order (3/3) 5S แจ้งเลขติดตาม → 7S ตัดสต๊อก → จัดส่งสำเร็จ



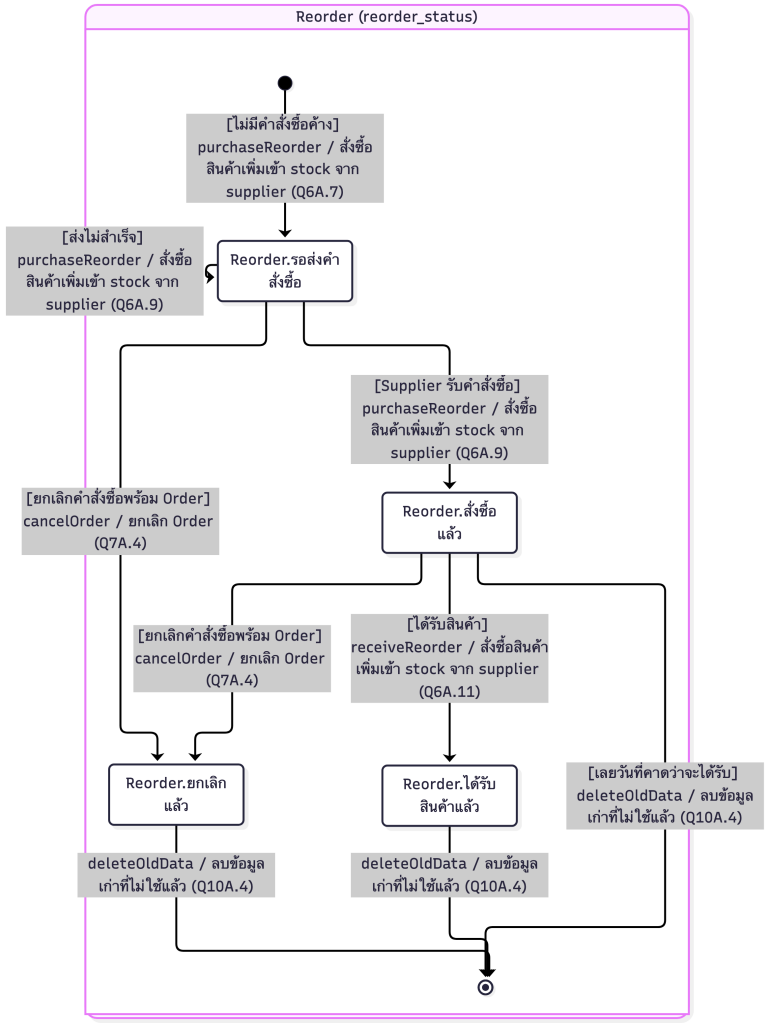
'จัดส่งสำเร็จ' ต้องครบ 3 เงื่อนไข

ขนส่งแจ้งส่งถึง (delivered) + แจ้งเลขติดตามแล้ว + ตัดสต๊อกครบทุกรายการ

ยกเลิกหลังส่งมอบ

7A → รอดำเนินการพิเศษ ให้ Admin จัดการต่อ

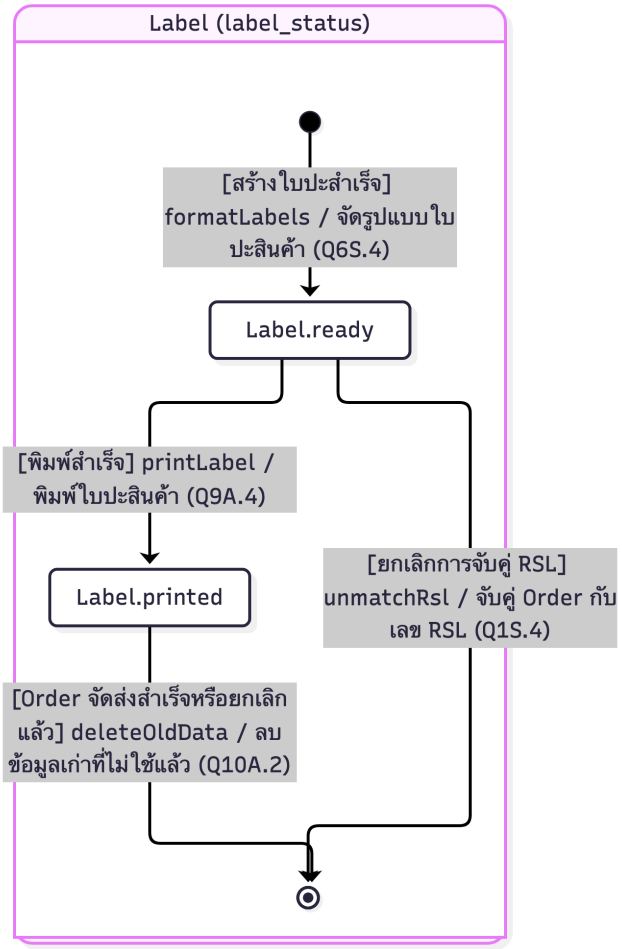
State diagram: Reorder สถานะใบสั่งซื้อ Supplier (reorder_status) · 5A, 6A, 7A, 10A



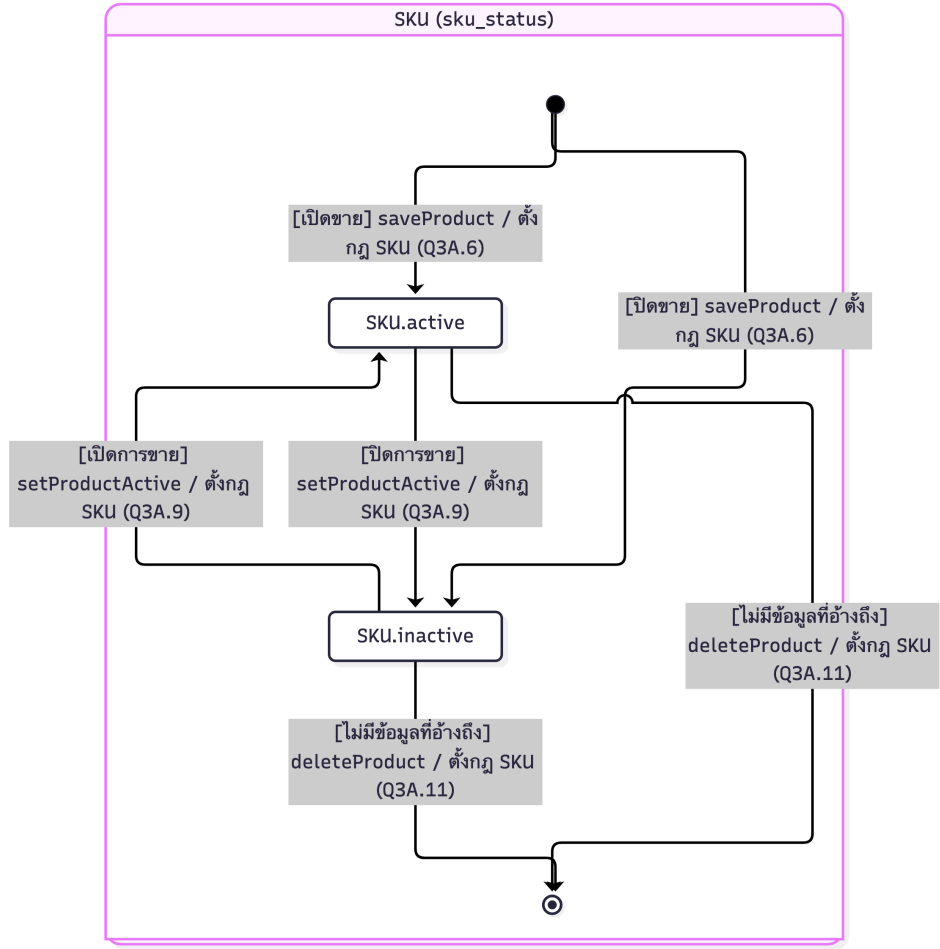
1. **Q6A.7 สร้าง**
   รอส่งคำสั่งซื้อ
2. **Q6A.9 Supplier รับ**
   สั่งซื้อแล้ว ส่งไม่สำเร็จคงสถานะเดิม
3. **Q6A.11 รับของ**
   ได้รับสินค้าแล้ว รับซ้ำไม่ได้
4. **Q7A.4 ยกเลิก**
   ยกเลิกแล้ว พร้อม Order
5. **Q10A.4 ลบ**
   เฉพาะใบที่ปิดแล้ว

State diagram: Label · SKU สถานะใบปะ (label_status) และสถานะสินค้า (sku_status)

สถานะใบปะ · 6S 9A 1S 10A

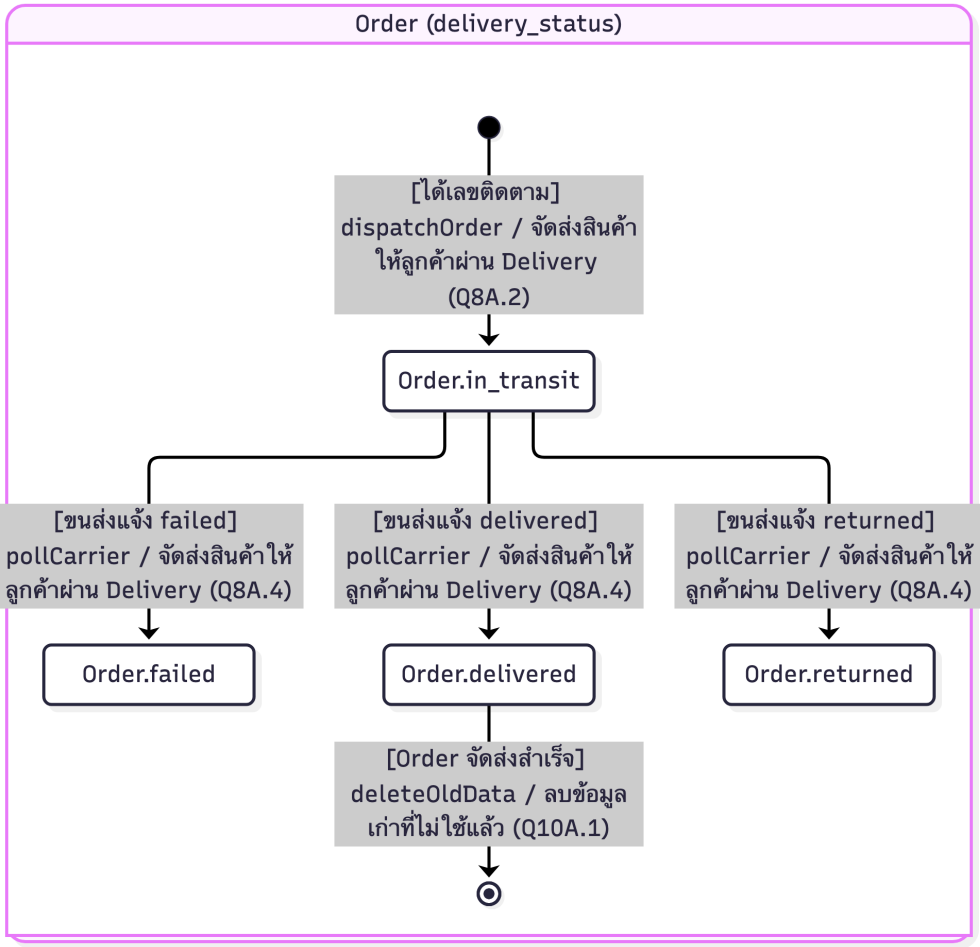


สถานะสินค้า (เปิด/ปิดขาย) · 3A

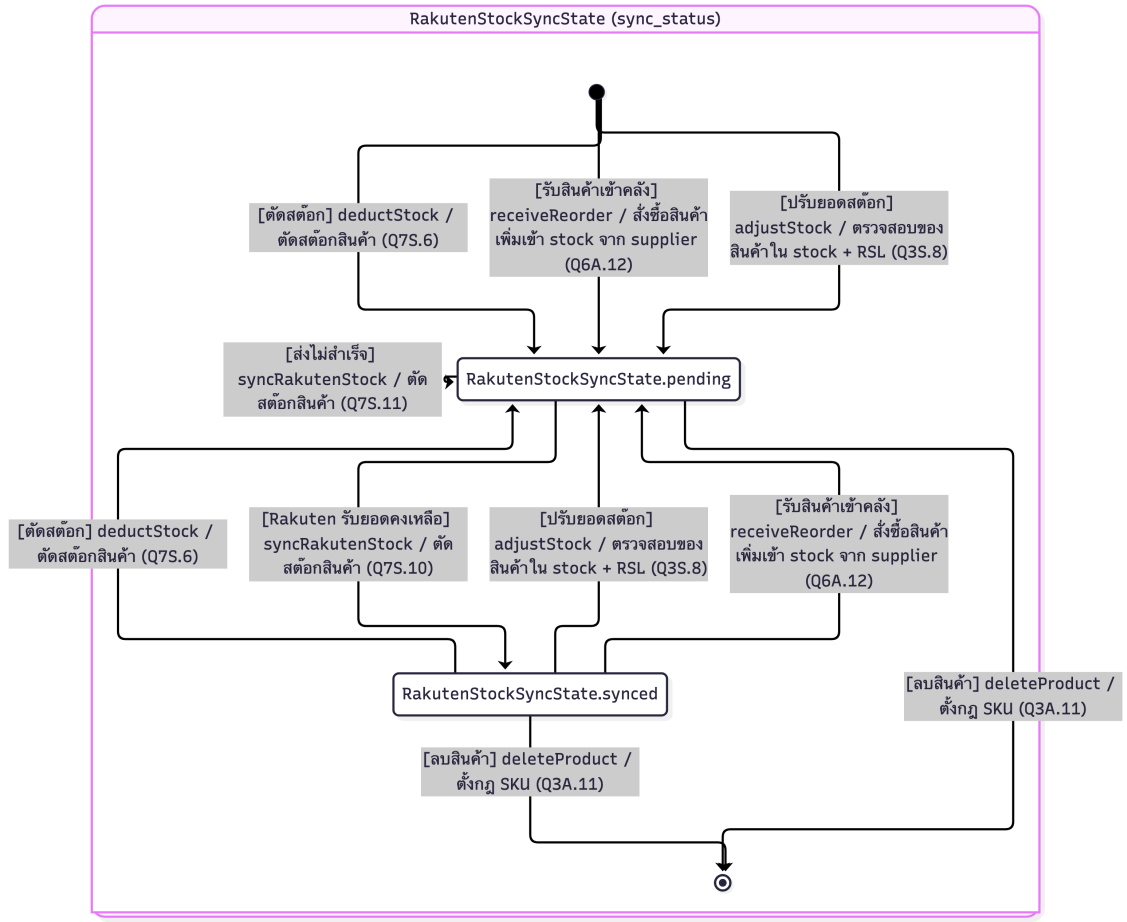


State diagram: delivery · sync สถานะขนส่ง และสถานะส่งยอดสต๊อกให้ Rakuten

สถานะขนส่ง (delivery_status) · 8A



ส่งยอดสต๊อกให้ Rakuten (sync_status) · 3S 6A 7S



Layers of the web app ทุกหน้าอ่านเขียน PostgreSQL จริง

View (Screen)	หน้าเว็บ (page)	`app/(app)/*`
↓		
Controller	รับคำสั่งจากหน้าเว็บ (Server Action)	`app/(app)/actions.ts`
↓		
UseCase	ขั้นตอนของแต่ละ Use Case	`lib/use-cases/*`
↓		
Queries	เก็บคำสั่ง SQL	`lib/queries/*`
↓		
DBConnector	ต่อฐานข้อมูล PostgreSQL	`lib/db.ts`

Adapters (ตัวจำลอง)

UseCase เรียกระบบภายนอกผ่านตัวเชื่อม (interface)

ไม่มีในโค้ด

- API แยกให้ระบบอื่นเรียก (REST API)
- คลาส Controller / Repository (เป็นชื่อบทบาท)
- ตัวตั้งเวลาทำงาน (Scheduler / cron)
- เก็บข้อมูลในเบราว์เซอร์ (ใช้แค่จำภาษา)

ฐานข้อมูล

PostgreSQL 17 ตาราง สร้างจากไฟล์ 001–008
รีเซ็ตเป็นข้อมูลตั้งต้นด้วย npm run db:reset

executeWorkflow Use Case อัตโนมัติทำต่อจากปุ่มของ Admin ในงานเดียวกัน (transaction)

Admin กดปุ่ม 1 ครั้ง ระบบเปิดงานเดียว (transaction) แล้วทำตามลำดับ

1. ทำสิ่งที่ Admin กด เช่น 2A ยืนยัน Order
2. ทำ Use Case อัตโนมัติต่อทันที (runAutomation)
   - Order ที่ส่งมา: 2S → 3S → 1S หรือ 6S
   - ลองใหม่: Order ค้าง 'รอจับคู่กฎ SKU' (Q2S.5)
   - ลองใหม่: Order ค้างตัดสต๊อก (Q7S.1)
   - ส่งยอดสต๊อกให้ Rakuten (7S)
3. สำเร็จ → บันทึกทั้งหมด (COMMIT)
   พัง → ยกเลิกทั้งหมด (ROLLBACK)

ถ้าขั้นอัตโนมัติพัง

- ยกเลิกทั้งหมด รวมสิ่งที่ Admin กด
- บันทึกข้อผิดพลาดแยกไว้ (auto_error_logs)
- ข้อมูลจึงไม่ค้างครึ่งทาง

ทำไมไม่มี Scheduler

ผลเห็นทันที

Admin กดยืนยันแล้วเห็นใบปะในหน้าถัดไป

ข้อมูลไม่ค้างครึ่งทาง

ปุ่มกับขั้นอัตโนมัติบันทึกพร้อมกัน

ไม่มีงานค้างตลอดไป

ทุกครั้งที่กดปุ่ม ระบบลองงานที่ค้างใหม่

เปิดหน้าจัดส่ง

ดึงสถานะขนส่ง และ 6S ทำงานต่อ

Locks and double deduction ล็อกแถว (row lock) กันแก้ข้อมูลชนกัน

ล็อกแถวก่อนแก้ (คำสั่ง FOR UPDATE)

Query	ล็อกข้อมูล	กันอะไร
Q2A.1	Order	Admin สองแท็บยืนยันพร้อมกัน
Q3S.1	สต๊อก	สอง Order จองของชิ้นเดียว
Q5A.1	Order	ตัดสินใจซ้ำ
Q6A.10	ใบสั่งซื้อ + สต๊อก	รับของซ้ำ
Q7S.2	รายการ + สต๊อก	ตัดสต๊อกซ้ำ
Q7S.1	ข้ามแถวที่ล็อก ≤100	รอบลองใหม่ไม่ต้องรอ

กันตัดสต๊อกซ้ำ 3 ชั้น

1. **ล็อกรายการ + สต๊อก (Q7S.2)**
   รอบที่สองต้องรอรอบแรกบันทึกเสร็จ
2. **ตัดเฉพาะรายการที่ยังไม่ตัด**
   โค้ดข้ามรายการที่ตัดแล้วก่อนเริ่ม
3. **แก้แบบมีเงื่อนไข (Q7S.5)**
   ตั้งเป็น 'ตัดแล้ว' ได้เฉพาะแถวที่ยังไม่ตัด

ยอดในคลังไม่ติดลบ

- Q7S.4 ลดยอดได้เฉพาะเมื่อยอดในคลังพอ
- สต๊อกไม่พอ → 'รอดำเนินการด้วยตนเอง' ไม่วนตัดซ้ำ

External systems ทุกระบบภายนอกต่อผ่านตัวเชื่อม (adapter) ตอนนี้เป็นตัวจำลอง

ตัวเชื่อม (adapter) 6 ตัว

Adapter	ใช้ใน	งาน
RakutenAdapter	4A 2A 5S 7S	ดึง Order, ส่งเลขติดตาม อัปเดตสต๊อก
RslAdapter	3S 1S	ยอดสต๊อก RSL, จับคู่เลข RSL
LabelServiceAdapter	6S	สร้างใบปะ
PrinterAdapter	9A	ส่งงานพิมพ์
CarrierAdapter	8A	เลขติดตาม, สถานะพัสดุ
SupplierAdapter	6A	ส่งใบสั่งซื้อ

จากตัวจำลองไปของจริง

1. **เขียนตัวเชื่อมของจริง**

   ตาม interface เดิม เช่นเรียก Rakuten RMS API

2. **เพิ่มโหมดในการตั้งค่า (AdapterConfig)**

   ตอนนี้มีโหมดเดียวคือ "mock" (จำลอง)

3. **UseCase ไม่ต้องแก้**

   เพราะเรียกผ่าน interface

จำลองระบบล่ม

- หน้าทดสอบ /dev (ไม่อยู่ในเมนู)
- สั่งให้ตัวเชื่อมตอบผิดพลาดได้
- ใช้ทดสอบกรณีผิดพลาดของทุก Use Case

Security รหัสผ่าน session และ SQL

รหัสผ่าน

- ไม่เก็บรหัสผ่านตรงๆ
- เข้ารหัสทางเดียวด้วย scrypt
- สุ่ม salt 16 byte ทุกบัญชี
- เทียบรหัสแบบใช้เวลาเท่ากันเสมอ (timingSafeEqual)

ค่าตั้ง scrypt: N=16384, r=8, p=1 · key 64 byte

Session

- บัตรผ่าน (token) สุ่ม 32 byte เก็บใน cookie
- ฐานข้อมูลเก็บแค่ค่า hash (SHA-256)
- JavaScript อ่าน cookie ไม่ได้ (httpOnly)
- อายุ 8 ชั่วโมง ออกจากระบบแล้วใช้ต่อไม่ได้

cookie: sameSite=lax · secure ตอน production · ออกจากระบบ = ตั้ง revoked_at

ฐานข้อมูลและบัญชี

- SQL ทุกข้อส่งค่าแยก ($1, $2) กัน SQL injection
- รหัสผิด 5 ครั้ง ล็อก 15 นาที
- อีเมลผิดหรือรหัสผิด ได้ข้อความเดียวกัน
- ทุกหน้าหลัง Login ต้องมี session

Platform and components เครื่องมือที่ใช้ และ Component Diagram

Software Development Platform

เครื่องมือที่ใช้พัฒนา RSL Fulfillment Hub (Colorado Co., Ltd. เฟส 1)

Frontend หน้าจอของ Admin	**Next.js 16** App Router · **React 19** · **TypeScript 5**
UI หน้าตาและ component	**Tailwind CSS 4** · **shadcn/ui** Radix UI · **TanStack Table** ตารางข้อมูล · **Recharts** กราฟ Dashboard · **Sonner** ข้อความแจ้งเตือน · **Lucide** ไอคอน · **Noto Sans Thai** ฟอนต์
Database ที่เก็บข้อมูล	**PostgreSQL** ผ่าน node-postgres (pg)
ระบบภายนอก เชื่อมผ่าน API	**Rakuten RMS API** ดึง Order, ส่งเลขติดตาม, อัปเดตสต็อก · **RSL** Rakuten Super Logistics · **ผู้ให้บริการขนส่ง** สถานะพัสดุ
เครื่องมือพัฒนา เขียน ตรวจ และเก็บโค้ด	**Node.js** npm · **ESLint** · **Git / GitHub** · **Google Chrome** ทดสอบ · **draw.io** Business Process, Use Case · **Mermaid** Sequence, Collaboration

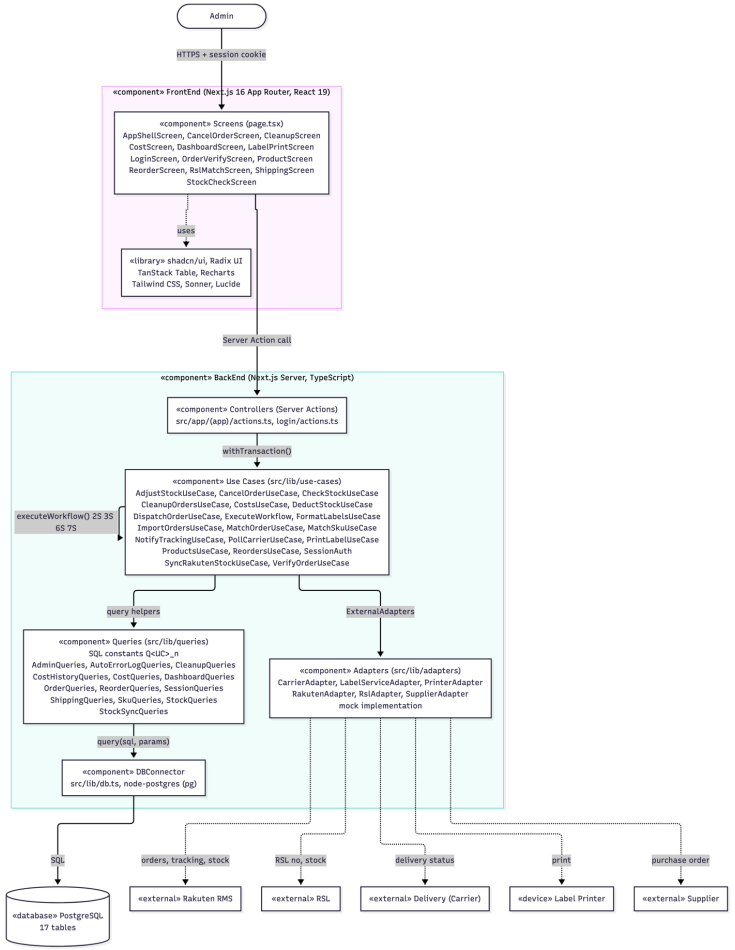
หน้าเว็บ: Next.js 16, React 19, TypeScript 5

หน้าตา: Tailwind CSS 4, shadcn/ui

ฐานข้อมูล: PostgreSQL (ต่อผ่าน node-postgres)

รัน SQL ในเล่มด้วย PGlite (PostgreSQL ในหน่วยความจำ)

Component Diagram

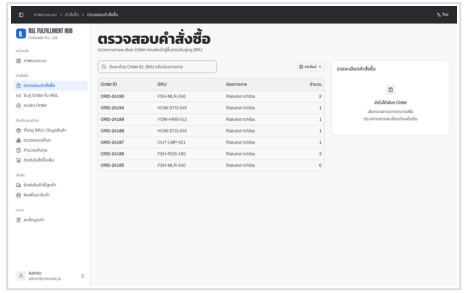


Site map and screens เริ่มที่หน้า Login · ชื่อหน้าจอตรงกับ Screen ใน Sequence

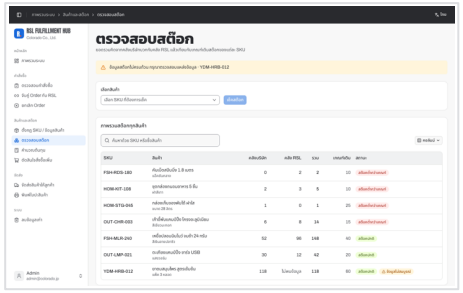
Site Map

```
LoginScreen (1A)
└ DashboardScreen (4A)
  ├ คำสั่งซื้อ
  │ ├ OrderVerifyScreen (2A)
  │ ├ RslMatchScreen (1S)
  │ └ CancelOrderScreen (7A)
  ├ สินค้าและสต๊อก
  │ ├ ProductScreen (3A)
  │ ├ StockCheckScreen (3S)
  │ ├ CostScreen (4S)
  │ └ ReorderScreen (5A, 6A)
  ├ จัดส่ง
  │ ├ ShippingScreen (8A, 5S)
  │ └ LabelPrintScreen (9A)
  └ ระบบ
    └ CleanupScreen (10A)
```

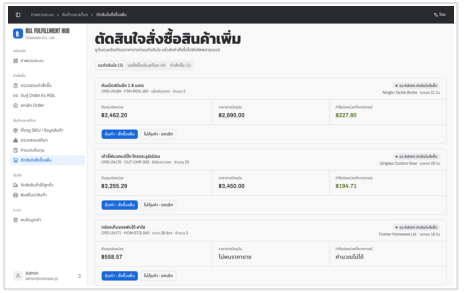
OrderVerifyScreen · 2A



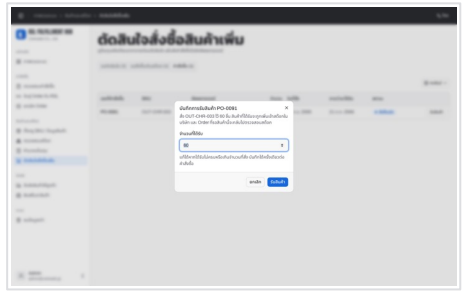
StockCheckScreen · 3S



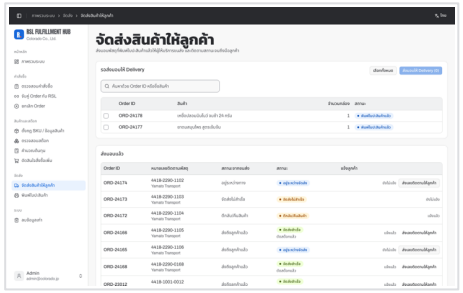
ReorderScreen · 5A 6A



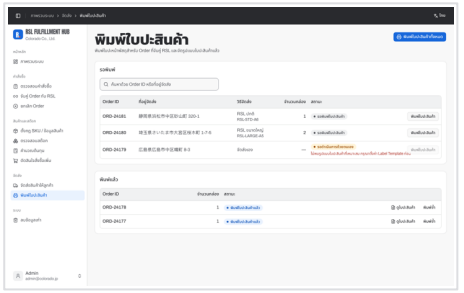
ReorderScreen · รับสินค้า



ShippingScreen · 8A 5S

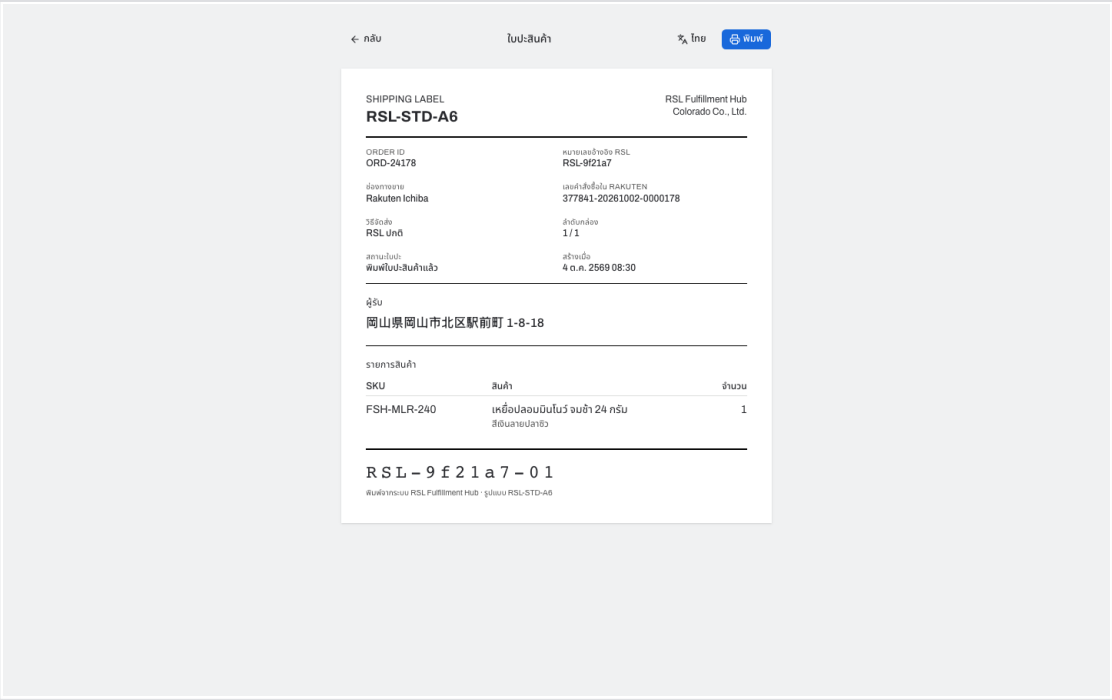


LabelPrintScreen · 9A

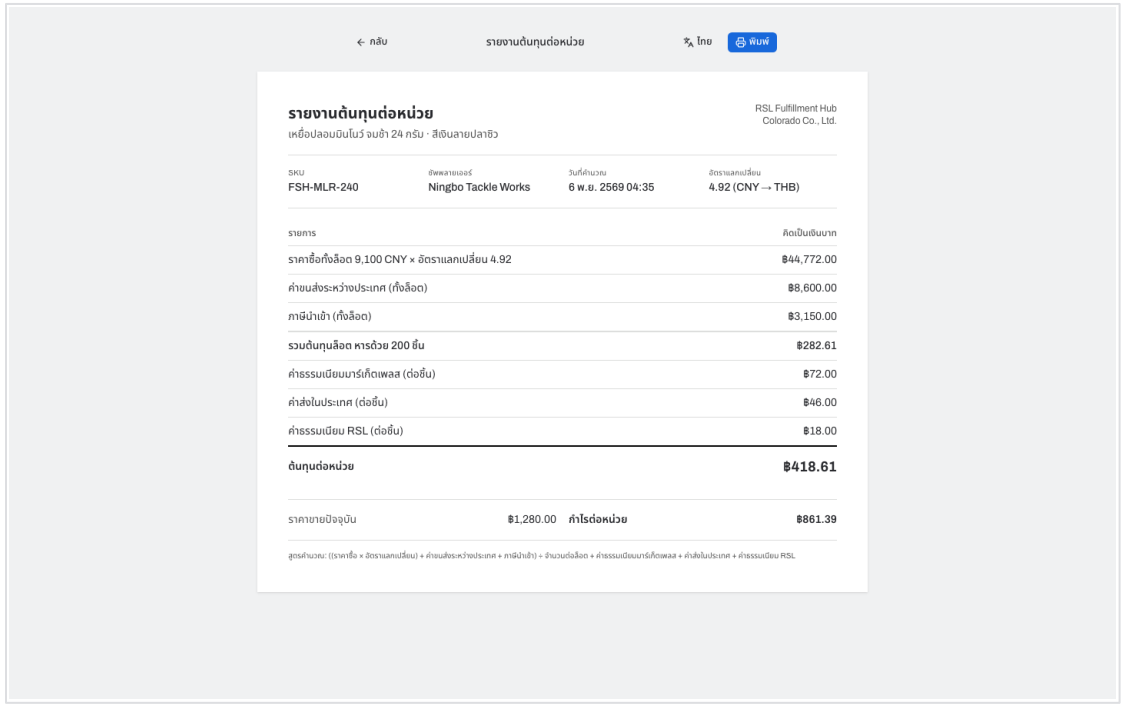


Output design รายงาน 2 ฉบับจากข้อมูลจริง

ใบปะสินค้า (Shipping Label)



รายงานต้นทุนต่อหน่วย (Unit Cost Report)



สร้างจาก 6S · พิมพ์จาก 9A

- แบบใบปะ 3 แบบ ตามแหล่งส่งและวิธีส่ง
- แยกพัสดุหลายกล่อง (1/2, 2/2) ภายใต้ Order เดียว

คำนวณจาก 4S

- องค์ประกอบต้นทุน 8 รายการ แปลงหยวน (CNY) เป็นบาท
- เก็บประวัติทุกครั้งที่คำนวณ ใช้ประกอบการตัดสินใจ 5A

SQL results ทุกคำสั่งรันกับข้อมูลตั้งต้นชุดเดียวกัน

113/113

SQL รันผ่านทุกข้อ ไม่มีข้อผิดพลาด

คำสั่งขึ้นต้นด้วย	จำนวน
SELECT	**50**
INSERT	**15**
UPDATE	**30**
DELETE	**2**
WITH	**16**

WITH = หลายขั้นในคำสั่งเดียว
เช่น แก้สต๊อก แล้วบันทึกประวัติ

ตัวอย่างผล (2.23) · Q7S.5 ทำเครื่องหมายว่าตัดสต๊อกแล้ว

```
UPDATE order_items
SET stock_deducted = TRUE, deducted_at = CURRENT_TIMESTAMP
WHERE order_item_id = ANY($1::text[]) AND stock_deducted = FALSE
```

ค่าที่ส่งเข้า (params): ["{ORD-24174-ITEM-1}"]

ผลลัพธ์: 1 แถวถูกแก้ไข

1. **รันด้วยสคริปต์ในโฟลเดอร์ของเล่ม**
   คำสั่ง npm run sql รันทุก query บน PGlite (PostgreSQL)
   เริ่มจากข้อมูลตั้งต้นชุดเดียวกัน

2. **SQL เดียวกับในโค้ด**
   คัดลอกจากโค้ดของเว็บทุกข้อ และจดชื่อคู่กันไว้
   เทียบทีละข้อแล้วตรงกัน

Testing สิ่งที่ทดสอบแล้ว และสิ่งที่ยังไม่มี

36/36
กรณีทดสอบบนเว็บจริง
ผ่านทั้งหมด

17 UC
ครบทุก Use Case
ทั้งกรณีปกติและผิดพลาด

ตรวจอัตโนมัติในโค้ด

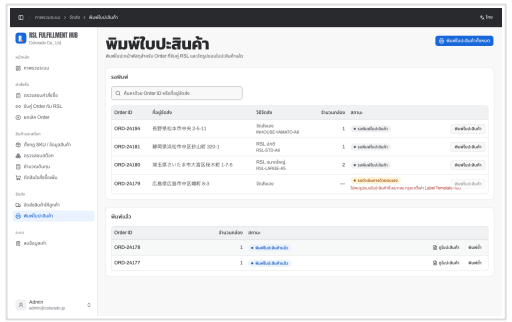
- ตรวจรูปแบบโค้ด ชนิดข้อมูล และ build ผ่าน
- ลองสร้างฐานข้อมูลใหม่ในที่ชั่วคราว ได้ครบ 17 ตาราง
- SQL 113 query รันผ่านสคริปต์ในโฟลเดอร์ของเล่ม

ยังไม่มี

ทดสอบอัตโนมัติแบบ unit test และ e2e test
ทดสอบหน้าจอ 36 กรณีด้วยมือ ตามตารางใน 2.24

UC	กรณีทดสอบ	ผล
1A	ใช้บัญชีที่เข้าผิดครบ 5 ครั้ง (locked@colorado.jp)	ผ่าน
3S	ยืนยัน ORD-24194 (RSL 0 ชิ้น, คลังบริษัท 1 ชิ้น)	ผ่าน
5A	เลือก "คุ้มค่า" ของ ORD-24184	ผ่าน
6A	รับสินค้า PO-0091 จำนวน 60 ชิ้น	ผ่าน
8A	ดึงสถานะจากขนส่งไม่ได้	ผ่าน
7S	อัปเดตจำนวนใน Rakuten ไม่ได้	ผ่าน

3S บริษัทส่งเอง · ORD-24194



Live demo + SELECT กดในเว็บ แล้ว SELECT ใน psql ให้เห็นข้อมูลเปลี่ยนจริง

#	กดในเว็บ	ผลที่ต้องเห็นหลัง SELECT
1	1A Login admin@colorado.jp	sessions 0 → 1 แถว (เก็บ token_hash)
2	4A นำเข้า Order ที่ Dashboard	รอตรวจสอบคำสั่งซื้อ 7 → 8
3	2A ยืนยัน ORD-24194	รอพิมพ์ใบปะสินค้า, in_house, label ready
4	9A พิมพ์ใบปะ ORD-24194	label printed + label_print_history 1 แถว
5	8A ส่งมอบ ORD-24194	อยู่ระหว่างจัดส่ง, in_transit (ยังไม่ตัดสต๊อก)
6	5A คุ้มค่า ORD-24184	รอสั่งซื้อจาก Supplier

ก่อนพรี (พิมพ์ใน terminal)

`docker start rsl-pg` เปิดฐานข้อมูล
`npm run db:reset` ล้างเป็นข้อมูลตั้งต้น `npm run dev` เปิดเว็บ
พลาด → กด reset ที่หน้า /dev แล้วเริ่มใหม่ · เวลาไม่พอ ทำข้อ 1–3

psql = โปรแกรมพิมพ์คำสั่ง SQL ของ PostgreSQL

```
-- เปิด psql เข้าฐานข้อมูลของเว็บ
docker exec -it rsl-pg psql \
  -U rsl_app -d rsl_fulfillment_hub
-- 1 session หลัง Login
SELECT admin_id, expires_at, revoked_at
FROM sessions;
-- 2 นับ Order ตามสถานะ
SELECT order_status, COUNT(*) FROM orders
GROUP BY 1 ORDER BY 2 DESC;
-- 3 4 5 สถานะ แหล่งส่ง ขนส่ง ใบปะ ของ ORD-24194
SELECT o.order_id, o.order_status,
  i.fulfill_source, o.tracking_number,
  o.delivery_status, l.label_status
FROM orders o JOIN order_items i USING (order_id)
LEFT JOIN labels l USING (order_id)
WHERE o.order_id = 'ORD-24194';
-- 6 Order ที่รอตัดสินใจ / รอสั่งซื้อ
SELECT order_id, order_status FROM orders
WHERE order_status IN ('รอ Admin ตัดสินใจสั่งซื้อ',
  'รอสั่งซื้อจาก Supplier');
```

Team and responsibilities หน้าที่รับผิดชอบของสมาชิก



นายธนภัทร พูลโภคา

6710450911

หัวหน้าทีม

งานที่รับผิดชอบ

- Business Flow As-Is / To-Be
- ER Diagram
- CRUD (As-Is)
- Use Case Description
- Backend และ Database

16 งานในเล่ม (ชิ้น)

5/5 คะแนนจากหัวหน้าทีม



นายอชิระ ภู่รุ่งเรือง

6710451488

งานที่รับผิดชอบ

- Business Flow As-Is / To-Be
- ER Diagram
- CRUD (To-Be), Query Table
- Use Case Description
- Backend และ Database

17 งานในเล่ม (ชิ้น)

5/5 คะแนนจากหัวหน้าทีม



นางสาวปิยากร ผลพานิช

6710451054

งานที่รับผิดชอบ

- ประสานงานกับเจ้าของกิจการ
- ผู้จัดทำรายงานหลัก
- Use Case Diagram
- Sequence, Collaboration
- UX/UI และ Frontend

20 งานในเล่ม (ชิ้น)

5/5 คะแนนจากหัวหน้าทีม

Summary สรุป

Before

- งานมือประมาณ 1–2 ชม./วัน
- คำนวณต้นทุน 20–30 นาที/ครั้ง
- ตัดสินใจสั่งซื้อด้วยความรู้สึก

After

- ระบบทำ 7 Use Case เอง
- ใบปะพร้อมทันทีหลังยืนยัน
- ต้นทุนจากสูตรเดียว
- เจ้าของยังอนุมัติทุกการสั่งซื้อ

Next

- ต่อ Rakuten RMS API และ RSL จริง
- เพิ่มตัวตั้งเวลาและทดสอบอัตโนมัติ
- ขยายไป Yahoo! Auctions และ Amazon

ระบบรับงานซ้ำ แต่ทุกการสั่งซื้อที่ใช้เงินจริงต้องผ่านการอนุมัติของเจ้าของ

17	17	113/113	36/36
Use Case Description	ตารางในฐานข้อมูล	SQL รันผ่าน	กรณีทดสอบผ่าน

Q&A.

ขอบคุณครับ/ค่ะ ยินดีตอบคำถาม

Colorado Back-Office System

นายธนภัทร พูลโภคา 6710450911

นายอชิระ ภู่รุ่งเรือง 6710451488

นางสาวปิยากร ผลพานิช 6710451054

01

Auto label

ใบปะพร้อมพิมพ์ทันที
เมื่อรู้ว่าส่งจาก RSL หรือคลังบริษัท

02

Auto reorder

เช็กสต๊อก 2 คลัง
เจ้าของอนุมัติสั่งซื้อและรับของ

03

Unit cost

ต้นทุนต่อหน่วยจากสูตรเดียวกันทุกครั้ง
พร้อมเก็บประวัติ